Vishal Acharya

1. Which of the following is best suitable chart to show data correlation?

Scatter

2.What type of data is best suited for box plots?

Continuous numerical data

3. In a box plot, the bottom line of the box represents which quartile?

First quartile

4. In a box plot, the top line of the box represents which quartile?

Third quartile

5. What is a heatmap used for?

To visualize numerical data in a grid-like format

6. Which parameter in the sns.heatmap() function is used to show numerical values in heatmap?

annot

7. What is the purpose of the cbar parameter in the sns.heatmap() function?

To add a colorbar to the heatmap

8. Which of the following types of graphs is not supported by NetworkX?

Vishal Acharya

Hypergraphs

9. Which of the following methods is used to add nodes to a graph in NetworkX?

graph.add_node()

10. Which parameter in sns.heatmap() displays numeric values inside each cell?

annot

11. If linewidths=2 and linecolor='black' are set in heatmap, what happens?

Cell borders become thick black lines

91 Which parameter removes the color bar in heatmap?

cbar=False

92 In sns.boxplot(), what does hue parameter do?

Groups boxplots by category

93 Which parameter controls color in sns.boxplot()?

color

94 In sns.scatterplot(), style parameter controls what?

Marker shape

Vishal Acharya

95 Which parameter sets marker size in sns.scatterplot()?

S

96 What is the purpose of vmin and vmax in heatmap?

Fix color scaling range

97 Which barmode creates grouped bars in px.bar()?

group

98 Which barmode is used for stacked bars in px.bar()?

stack

99 Which barmode is used for diverging bars px.bar()?

relative

100 What does pattern_shape parameter do in px.bar()?

Adds pattern based on category

101 orientation='h' in px.bar() creates what?

Horizontal bars

102 Which function creates a 3D scatter plot?

Vishal Acharya

px.scatter_3d()

103 Which parameter controls bubble size in px.scatter()?

size

104 symbol parameter in scatter plot controls what in px.scatter()?

Marker shape

105 Which function creates scatter matrix?

px.scatter_matrix()

106 facet_col in scatter plot is used for what in px.scatter()?

Create multiple subplots

107 Which parameter shows markers in line chart?

markers=True

108 How to plot multiple lines in px.line()?

Use color parameter

109 color_discrete_sequence is used for what IN px.line()?

Set custom line colors

Vishal Acharya

110 Which parameter controls number of bins in px.histogram()?

Calculate sum per bin

111 histfunc='sum' in histogram does what in px.histogram()?

Calculate sum per bin

112 Which method shows percent and label in pie chart?

update_traces(textinfo='percent+label')

113 Which heatmap parameter controls color palette?

cmap

114 Setting annot=False in heatmap will:

Hide values

115 In sns.boxplot(), x parameter usually represents:

Category variable

116 Which seaborn function creates scatter plot?

sns.scatterplot()

117 In px.bar(), color parameter is used to:

Vishal Acharya

Group bars by category

118 px.bar() width parameter controls:

Figure width

119 In px.scatter(), symbol parameter is used to:

Change marker shape

120 Which function creates stacked histogram?

px.histogram()

121 In px.histogram(), color parameter does what?

Groups data by category

122 Which function creates simple line chart?

px.line()

123 markers=True in px.line() will:

Add markers on data points

124 Which parameter sets custom colors in px.pie()?

color_discrete_sequence

Vishal Acharya

125 In px.pie(), values parameter represents:

Numeric values

126 scatter_matrix is used to:

Compare multiple numeric variables

127 Which plotly function supports 3D plotting?

px.scatter_3d()

128 Which parameter adds extra information on hover?

hover_data

129 text_auto=True does what?

Adds automatic value labels

130 Which parameter in px.scatter() creates subplots by row?

facet_row

131 Which seaborn function supports hue parameter?

sns.scatterplot()

132 In heatmap, linewidths controls:

Vishal Acharya

Cell border thickness

133 Which plot type shows proportion of total?

Pie

134 px.scatter_matrix() requires which parameter?

dimensions

135 In px.bar(), text parameter is used for:

Manual label display

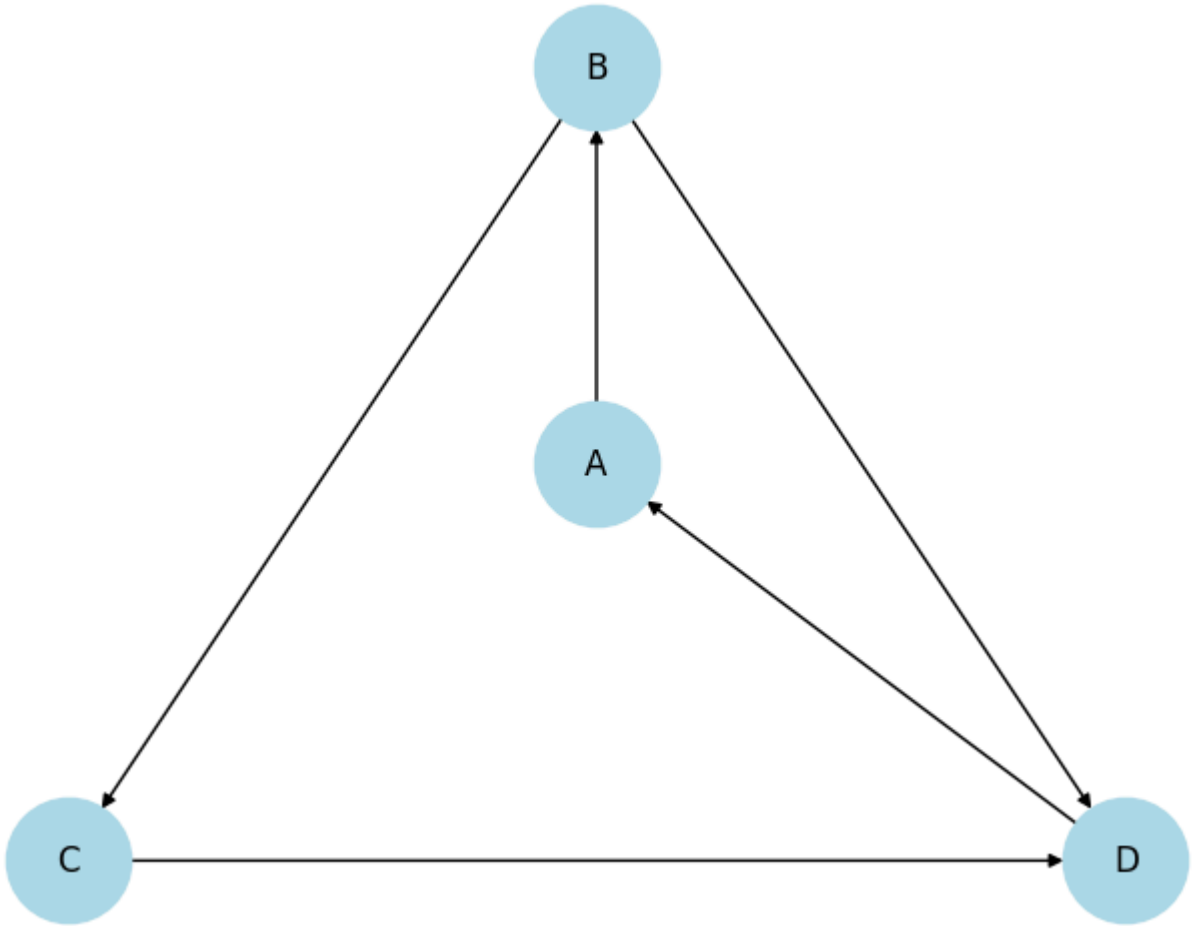
136 Write a python program which creates following graph using networkx module in python

In [1]:
```
import networkx as nx
import matplotlib.pyplot as plt

# Create Directed Graph
G = nx.DiGraph()

# Add nodes
G.add_nodes_from(["A", "B", "C", "D"])

# Add edges according to diagram
edges = [
    ("A", "B"),
    ("B", "C"),
    ("B", "D"),
    ("C", "D"),
    ("D", "A")
]

G.add_edges_from(edges)
```

Vishal Acharya

```python
# Custom positions to match the diagram layout
pos = {
    "B": (0, 2),
    "A": (0, 1),
    "C": (-1, 0),
    "D": (1, 0)
}

# Draw graph
nx.draw(
    G, pos,
    with_labels=True,
    node_size=2000,
    node_color="lightblue",
    arrows=True,
    font_size=12
)

plt.show()
```

Vishal Acharya



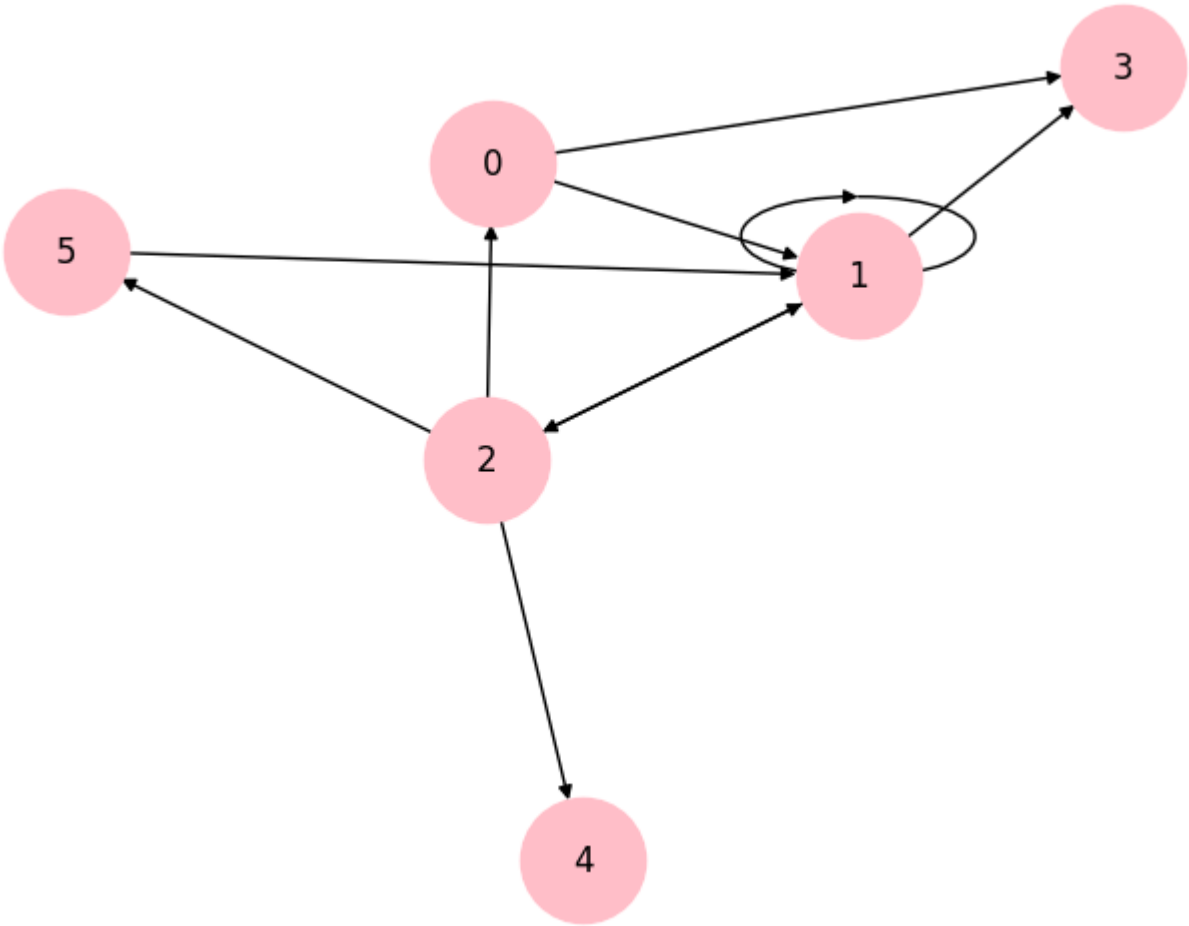
137 Using Networkx Library, Write a Code to create below given Directed Graph.

In [3]:
```
import networkx as nx
import matplotlib.pyplot as plt

# Create directed graph
G = nx.DiGraph()

# Add nodes
```

Vishal Acharya

```
G.add_nodes_from([0,1,2,3,4,5])

# Add edges according to diagram
edges = [
    (5,1),
    (2,5),
    (1,1),   # self loop
    (1,2),
    (2,1),
    (2,0),
    (0,1),
    (1,3),
    (0,3),
    (2,4)
]

G.add_edges_from(edges)



nx.draw(G,
        with_labels=True,
        node_color='pink',
        node_size=2000,
        arrows=True)

plt.show()
```

Vishal Acharya



138 Create a boxplot of the distribution of temperatures in different cities. Take data from 'temperatures.csv' from below:

In [5]:
```
%%writefile temp.csv
New York,Los Angeles,Chicago,Date
30,70,20,01-05-2023
35,72,22,02-05-2023
28,68,18,03-05-2023
32,75,24,04-05-2023
```

Vishal Acharya

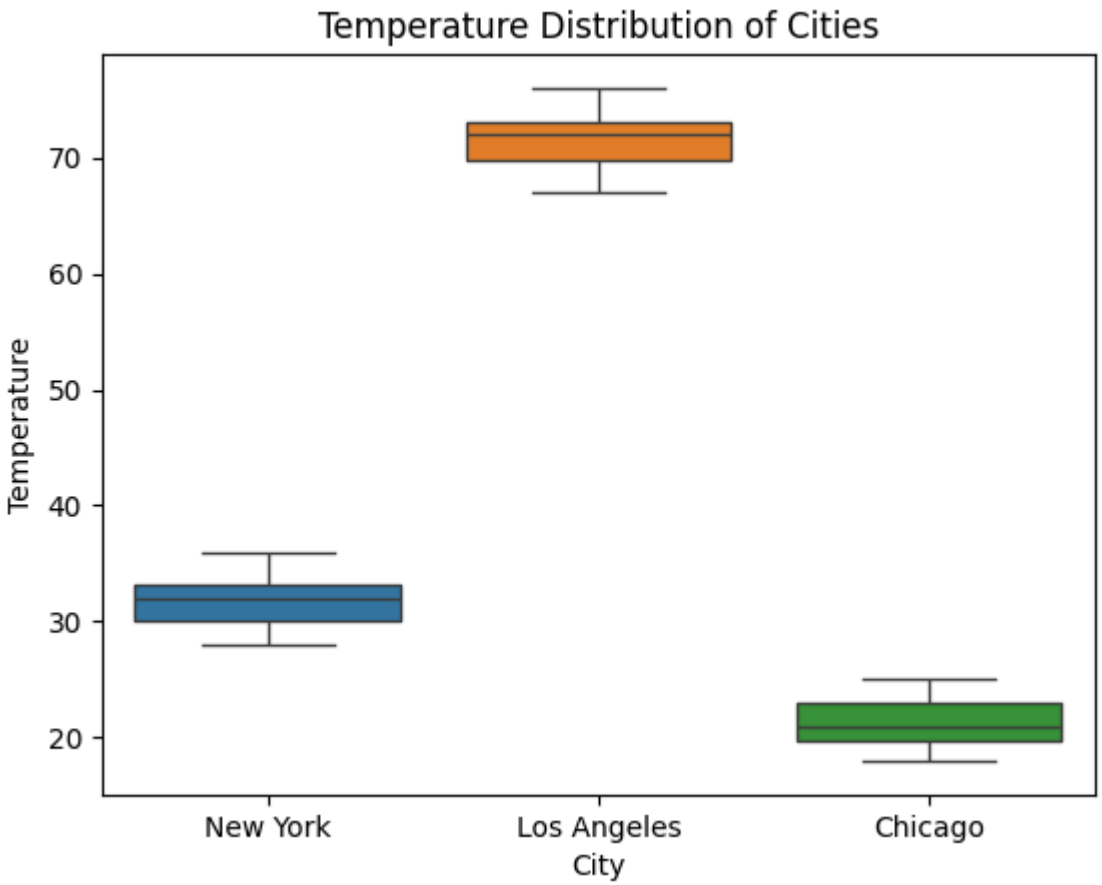
```
33,73,21,05-05-2023
29,71,19,06-05-2023
31,74,23,07-05-2023
34,72,20,08-05-2023
30,70,22,09-05-2023
33,69,25,10-05-2023
32,76,20,11-05-2023
36,72,18,12-05-2023
31,68,23,13-05-2023
29,73,19,14-05-2023
32,75,24,15-05-2023
34,69,21,16-05-2023
30,71,18,17-05-2023
35,73,22,18-05-2023
28,67,20,19-05-2023
32,72,23,20-05-2023
```

```
Writing temp.csv
```

In [7]:

```
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt

# Read CSV file
df = pd.read_csv("temp.csv")

# Create boxplot for city temperatures
sns.boxplot(data=df[["New York","Los Angeles","Chicago"]])

# Labels and title
plt.title("Temperature Distribution of Cities")
plt.xlabel("City")
plt.ylabel("Temperature")

plt.show()
```

Vishal Acharya



139 "The following dictionary shows how five people follow each other on Instagram:

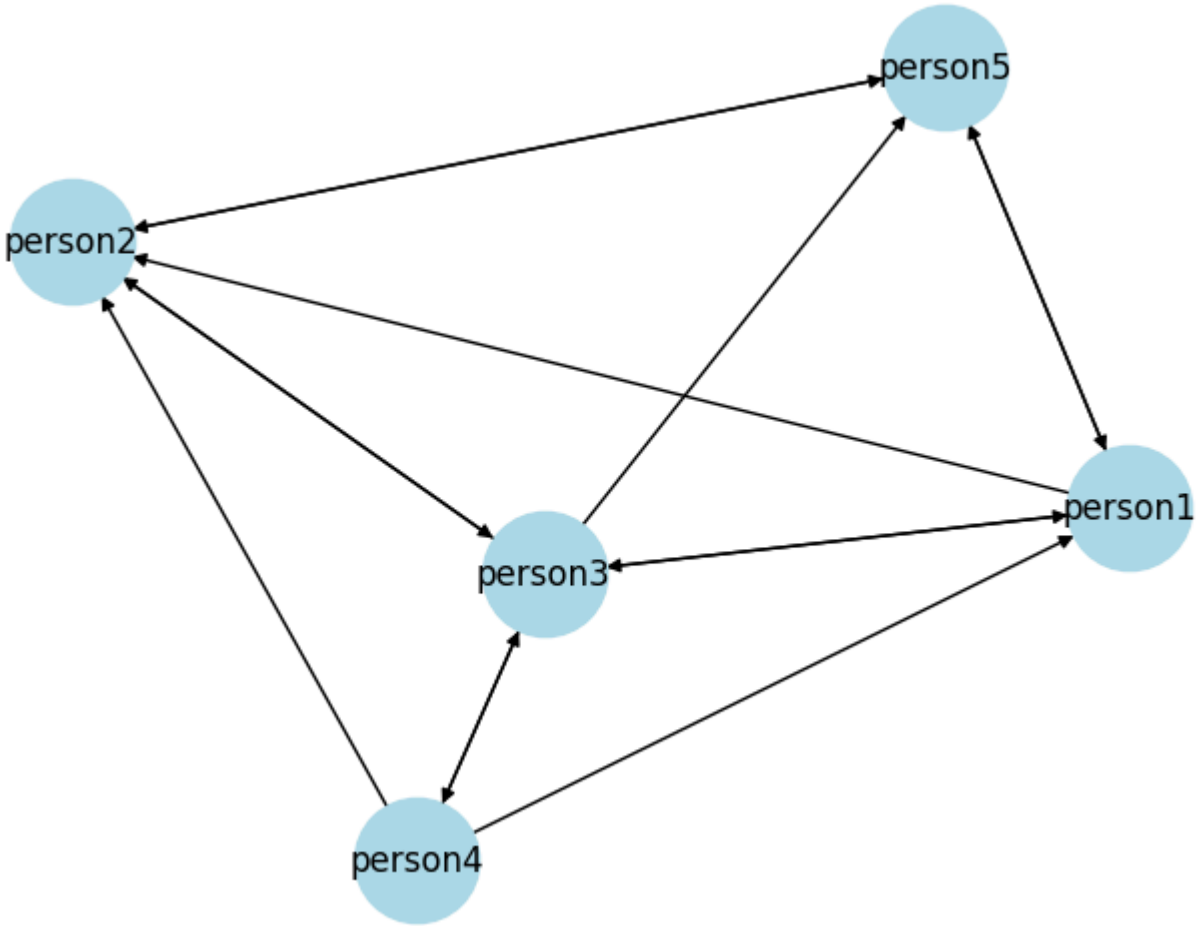
instagram = {'person1': [0,1,1,0,1], 'person2': [0,0,1,0,1], 'person3': [1,1,0,1,1],'person4': [1,1,1,0,0], 'person5': [1,1,0,0,0]} E.g., the list for person1 has the value on index 2 as 1 which means person1 followsperson3 and a directed edge should be added from person1 to person3. Using networkx library, create a directed graph.

In [8]:
```
import networkx as nx
import matplotlib.pyplot as plt

instagram = {
'person1': [0,1,1,0,1],
```

Vishal Acharya

```python
'person2': [0,0,1,0,1],
'person3': [1,1,0,1,1],
'person4': [1,1,1,0,0],
'person5': [1,1,0,0,0]
}

# Create directed graph
G = nx.DiGraph()

people = list(instagram.keys())

# Add edges
for person, follows in instagram.items():
    for i in range(len(follows)):
        if follows[i] == 1:
            G.add_edge(person, people[i])

# Draw graph
nx.draw(G, with_labels=True, node_color='lightblue', node_size=2000, arrows=True)

plt.show()
```

Vishal Acharya



140 You have been given a dataset of car prices and their respective horsepower, mileage, and weight. You have been tasked to analyze the relationship between these variables and create a scatter plot to visualize the patterns.

In [9]:
```
%%writefile car.csv
Price,Horsepower,Mileage,Weight
10000,120,25,2500
12000,140,20,3000
8000,100,30,2000
```

Vishal Acharya

```
15000,180,18,3500
9000,110,28,2400
13000,150,22,3200
11000,130,24,2800
17000,200,16,4000
7500,90,32,1800
14000,170,20,3300
9500,105,27,2200
10500,125,26,2600
12500,145,21,3100
15500,190,17,3800
8500,95,31,1900
11500,135,23,2700
16500,195,15,3900
7000,85,35,1700
14500,175,19,3400
9800,115,29,2300
13500,160,21,3000
11800,140,23,2700
12800,150,22,3100
8000,100,31,2000
9500,110,28,2400
```

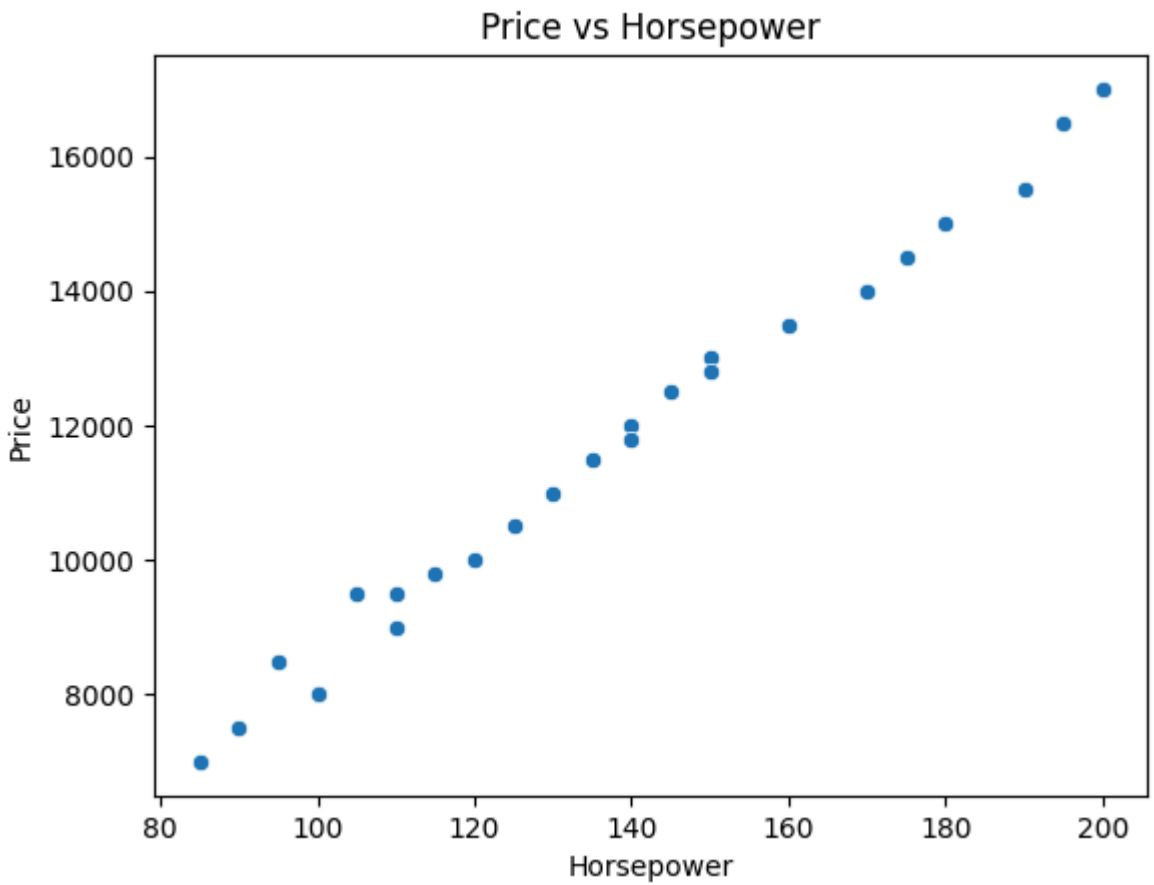
```
Writing car.csv
```

In [10]:

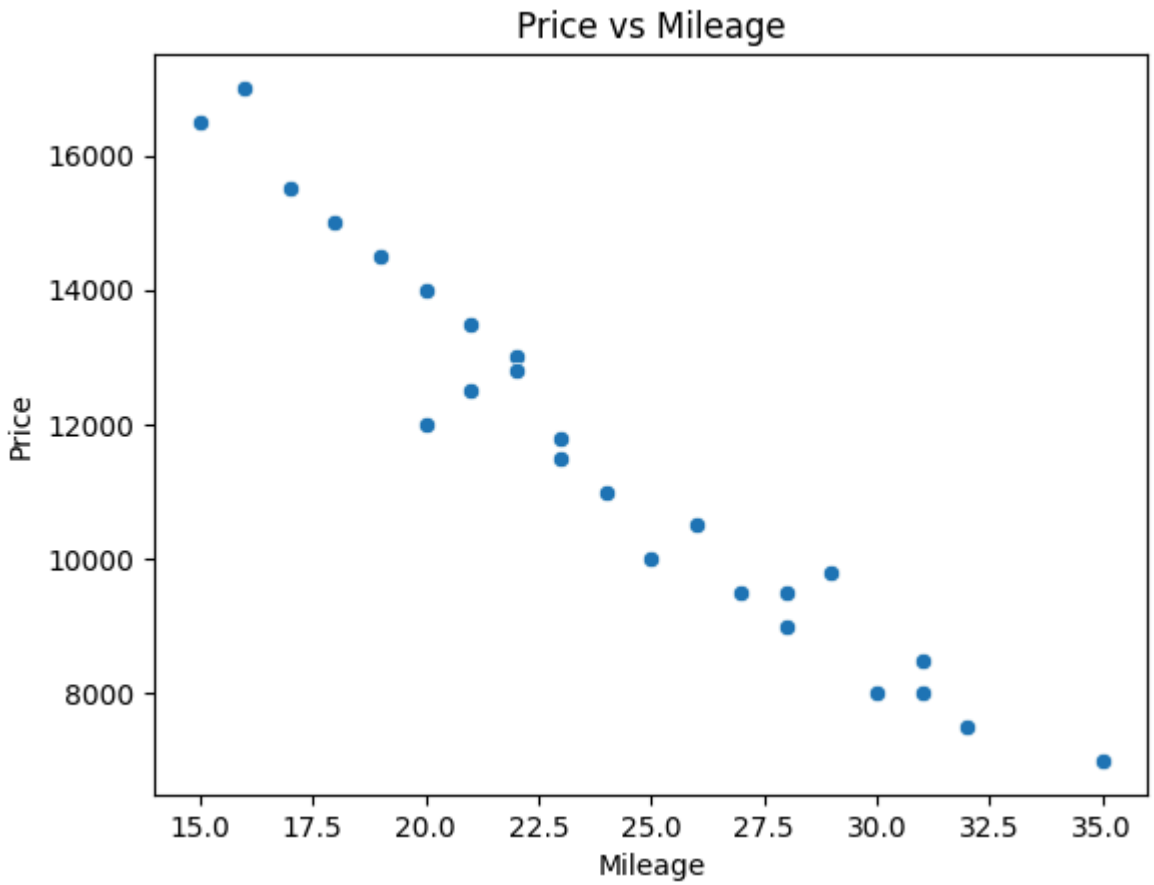
```python
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt

# Read the dataset
df = pd.read_csv("car.csv")

# Scatter plot: Price vs Horsepower
sns.scatterplot(x="Horsepower", y="Price", data=df)

plt.title("Price vs Horsepower")
plt.show()

# Scatter plot: Price vs Mileage
sns.scatterplot(x="Mileage", y="Price", data=df)
```

Vishal Acharya

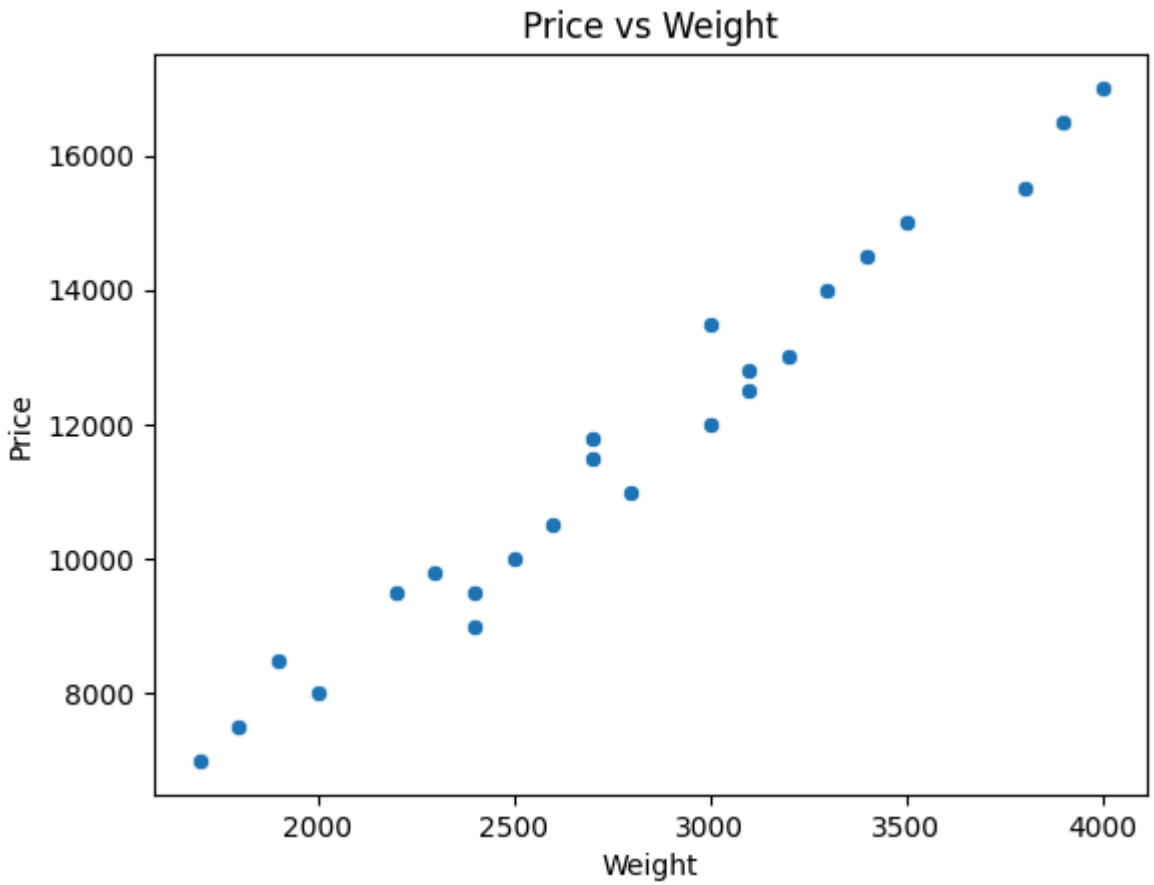
```
plt.title("Price vs Mileage")
plt.show()

# Scatter plot: Price vs Weight
sns.scatterplot(x="Weight", y="Price", data=df)

plt.title("Price vs Weight")
plt.show()
```



Vishal Acharya



Vishal Acharya



141 You have been given a dataset of house prices and their respective lot size and square footage. Your task is to create a scatter plot to determine if there is any correlation between these variables.

Dataset: The dataset, named "house_data.csv":

In [11]:
```
%%writefile house.csv
Price,LotSize,SqFt
5000000,5000,2000
7500000,7500,2500
4500000,5500,1800
```

Vishal Acharya

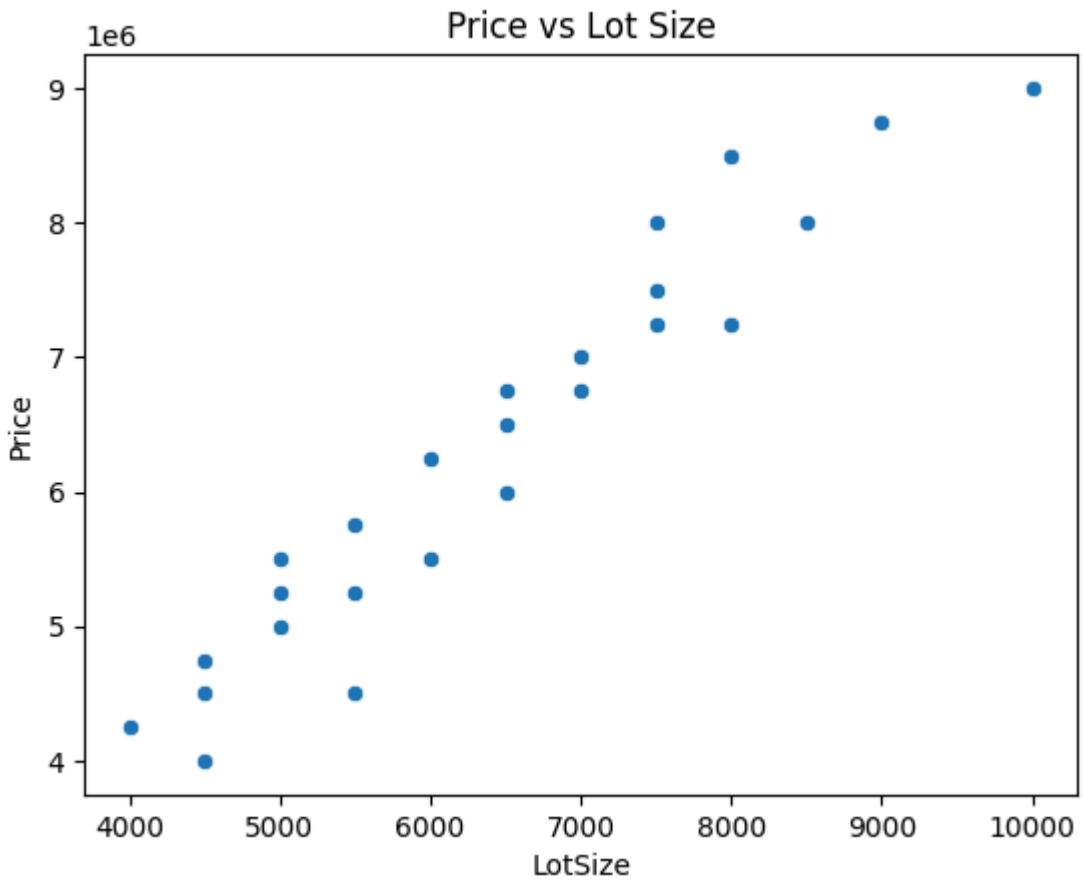
```
6000000,6500,2200
4000000,4500,1600
5500000,6000,2000
8500000,8000,3000
7000000,7000,2500
6500000,6500,2400
8000000,7500,2800
4500000,4500,1500
9000000,10000,3500
5750000,5500,1900
6250000,6000,2200
5250000,5000,1800
6750000,7000,2500
4750000,4500,1600
7250000,8000,2800
4250000,4000,1400
5250000,5500,2000
8000000,8500,3000
5500000,5000,1800
8750000,9000,3200
7250000,7500,2700
6750000,6500,2300
```

```
Writing house.csv
```

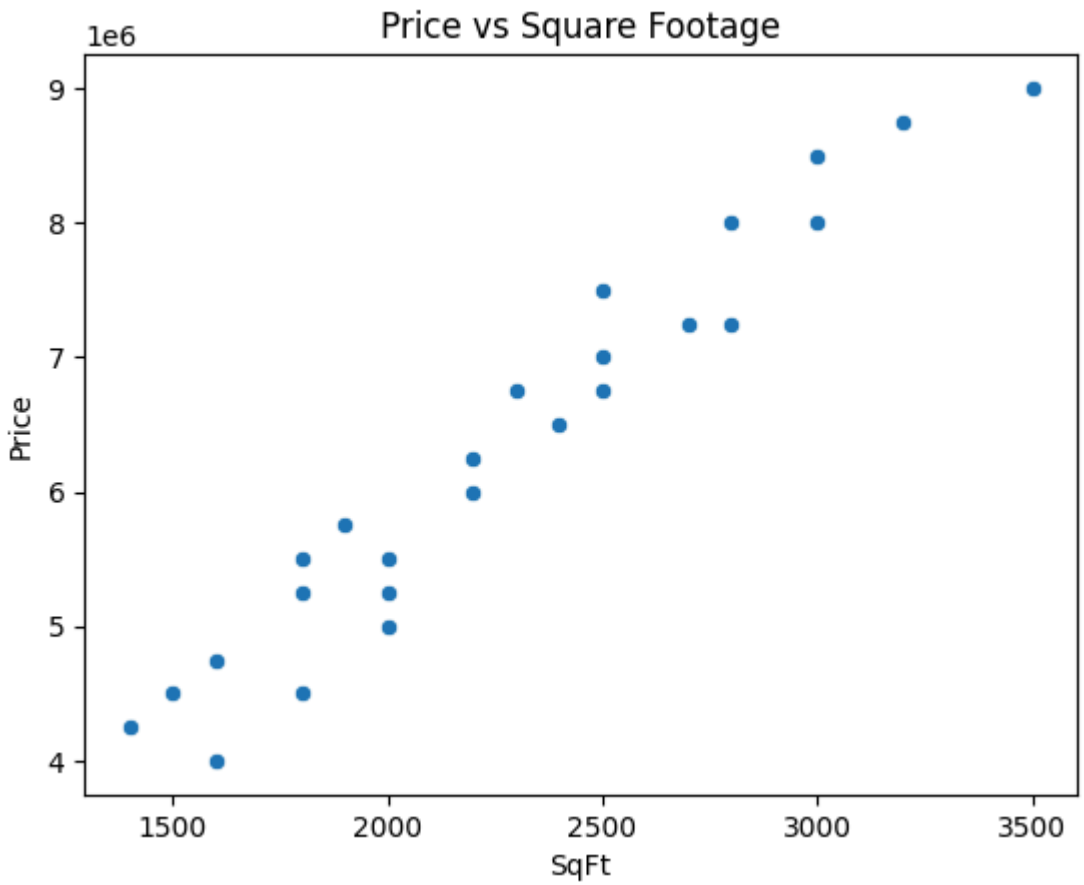
In [22]:

```python
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt

# Read dataset
df = pd.read_csv("house.csv")

# Scatter plot: Price vs Lot Size
sns.scatterplot(x="LotSize", y="Price", data=df)
plt.title("Price vs Lot Size")
plt.show()

# Scatter plot: Price vs Square Footage
sns.scatterplot(x="SqFt", y="Price", data=df)
plt.title("Price vs Square Footage")
plt.show()
```

Vishal Acharya



Vishal Acharya



142 "Use the file heights_weights.csv which contains 10000 non-null values for heights and weights. The Male column shows 1 if the person is a Male and 0 if the person is a Female. Take file of dataset from:

https://raw.githubusercontent.com/kavit88/Data-Sets/main/heights_weights.csv

1. Convert this file into a pandas Data Frame.
2. Display basic information like memory and data types for this data frame.
3. Display basic statistics like mean, std, quartiles, etc. for this data frame.
4. Create a correlation table for the data frame and comment about what kind ofcorrelation is there between Height and Weight.

Vishal Acharya

5. Do Height and Weight contain any outliers? Answer by creating boxplots for both.
6. Finally, create a scatter plot of Weight v/s Height with the following specifications:

"

In [15]:
```
import pandas as pd

url = "heights_weights.csv"

df = pd.read_csv(url)

print(df.head())
```

```
      Height      Weight  Male
0  73.847017  241.893563     1
1  68.781904  162.310473     1
2  74.110105  212.740856     1
3  71.730978  220.042470     1
4  69.881796  206.349801     1
```

Display basic information like memory and data types for this data frame.

In [16]:
```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 10000 entries, 0 to 9999
Data columns (total 3 columns):
 #   Column  Non-Null Count  Dtype
---  ------  --------------  -----
 0   Height  10000 non-null  float64
 1   Weight  10000 non-null  float64
 2   Male    10000 non-null  int64
dtypes: float64(2), int64(1)
memory usage: 234.5 KB
```

Display basic statistics like mean, std, quartiles, etc. for this data frame.

In [18]:
```
df.describe()
```

Vishal Acharya

Out[18]:

	Height	Weight	Male
count	10000.000000	10000.000000	10000.000000
mean	66.367560	161.440357	0.500000
std	3.847528	32.108439	0.500025
min	54.263133	64.700127	0.000000
25%	63.505620	135.818051	0.000000
50%	66.318070	161.212928	0.500000
75%	69.174262	187.169525	1.000000
max	78.998742	269.989699	1.000000

Create a correlation table for the data frame and comment about what kind ofcorrelation is there between Height and Weight.

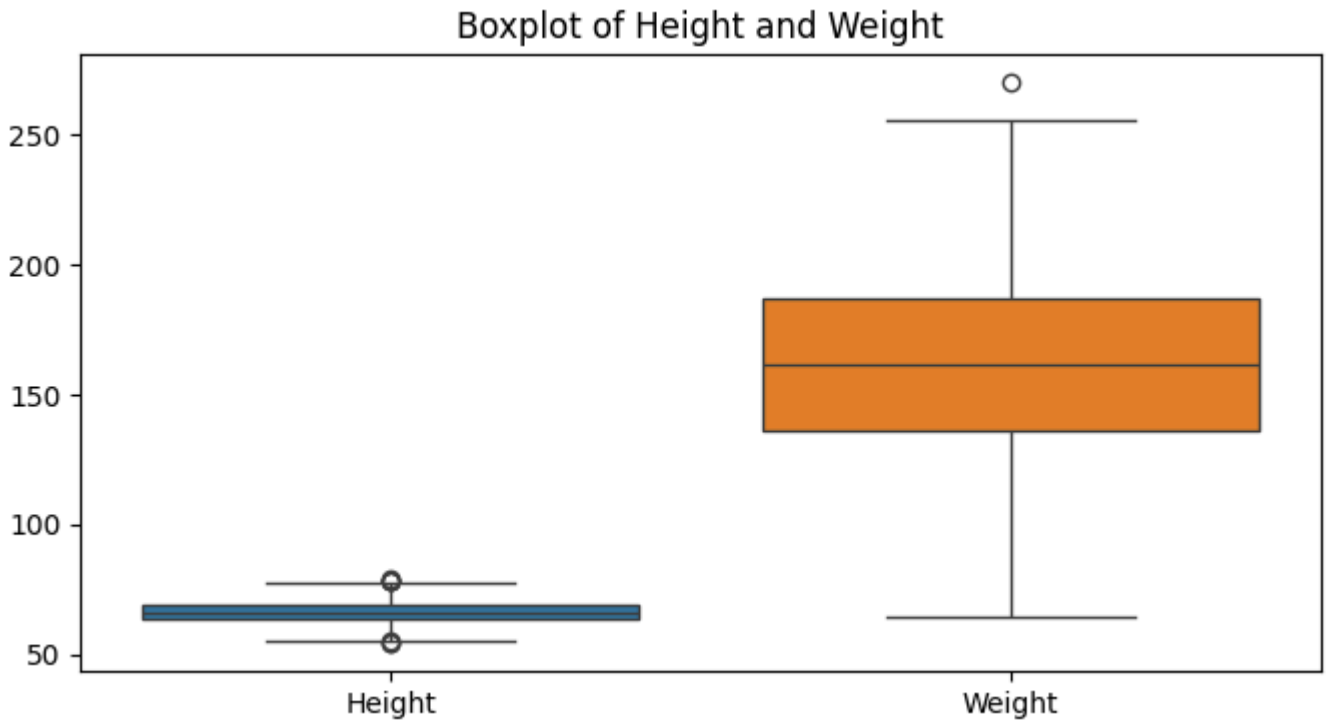
In [19]:
```
corr = df.corr(numeric_only=True)
print(corr)
```

```
          Height    Weight      Male
Height  1.000000  0.924756  0.691072
Weight  0.924756  1.000000  0.796723
Male    0.691072  0.796723  1.000000
```

Do Height and Weight contain any outliers? Answer by creating boxplots for both.

In [20]:
```
import seaborn as sns
import matplotlib.pyplot as plt

plt.figure(figsize=(8,4))
sns.boxplot(data=df[["Height","Weight"]])
plt.title("Boxplot of Height and Weight")
plt.show()
```

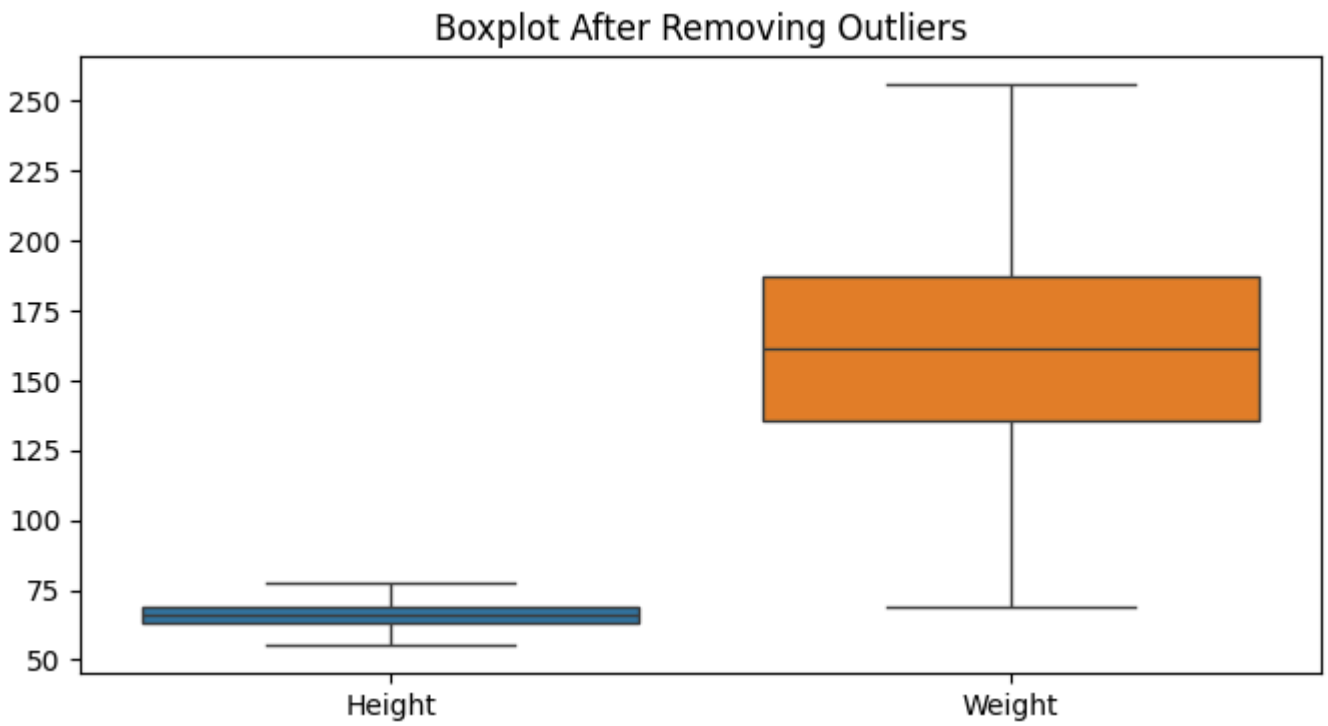
Vishal Acharya



In [22]:
```
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt

# Calculate Q1 and Q3
Q1 = df[["Height","Weight"]].quantile(0.25)
Q3 = df[["Height","Weight"]].quantile(0.75)

# Calculate IQR
IQR = Q3 - Q1

# Remove outliers
df_clean = df[~((df[["Height","Weight"]] < (Q1 - 1.5 * IQR)) |
                (df[["Height","Weight"]] > (Q3 + 1.5 * IQR))).any(axis=1)]

# Boxplot after removing outliers
plt.figure(figsize=(8,4))
```

Vishal Acharya

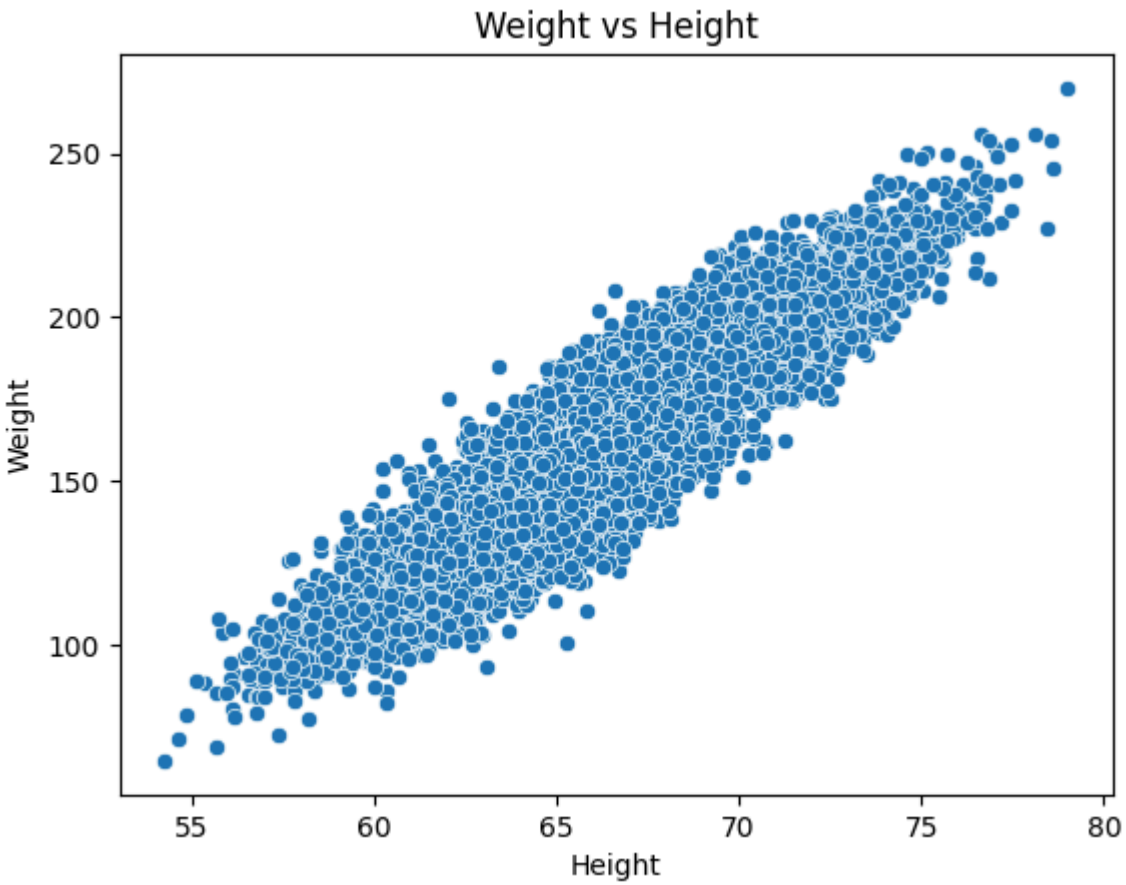
```
sns.boxplot(data=df_clean[["Height","Weight"]])
plt.title("Boxplot After Removing Outliers")
plt.show()
```



Finally, create a scatter plot of Weight v/s Height with the following specifications: "

In [21]:
```
sns.scatterplot(x="Height", y="Weight", data=df)

plt.title("Weight vs Height")
plt.xlabel("Height")
plt.ylabel("Weight")

plt.show()
```

Vishal Acharya

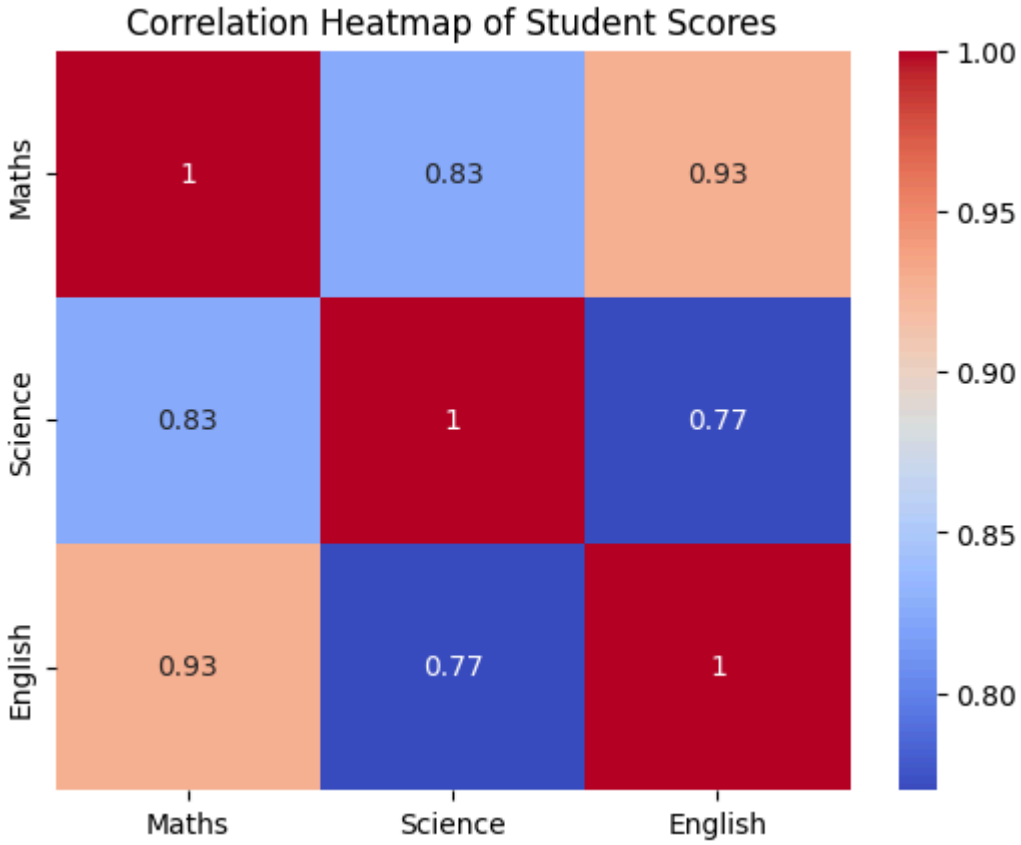


143 The file "student_scores.csv" contains the marks scored by a group of students in three subjects: Maths, Science, and English. Each row contains the name of the student, their score in Maths, Science, and English. Create a pandas DataFrame from this data and create a heatmap to visualize the correlations between the scores in these three subjects.

In [25]:
```
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt

# Read the dataset
df = pd.read_csv("student_scores (1).csv")

# Calculate correlation between subjects
```

Vishal Acharya

```
corr = df[["Maths","Science","English"]].corr()

# Create heatmap
sns.heatmap(corr, annot=True, cmap="coolwarm")

plt.title("Correlation Heatmap of Student Scores")
plt.show()
```



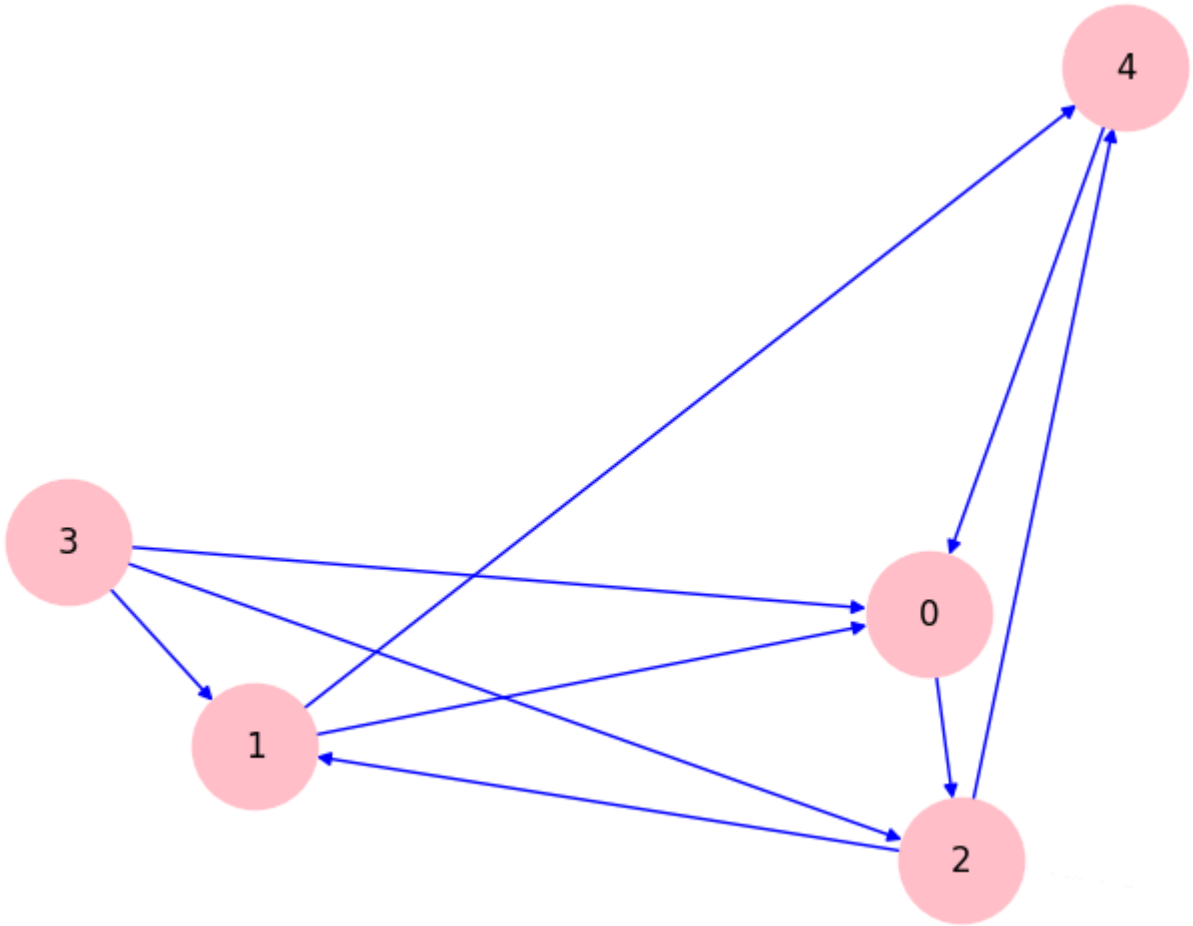
144 Using Networkx Library, Write a code to Create a below given Directed Graph.

In [26]:
```
import networkx as nx
import matplotlib.pyplot as plt
```

Vishal Acharya

```python
# Create directed graph
G = nx.DiGraph()

# Add nodes
G.add_nodes_from([0,1,2,3,4])

# Add edges based on the diagram
edges = [
    (3,1),
    (3,2),
    (3,0),
    (2,4),
    (2,1),
    (1,4),
    (1,0),
    (4,0),
    (0,2)
]

G.add_edges_from(edges)


nx.draw(
    G,
    with_labels=True,
    node_color='pink',
    node_size=2000,
    edge_color='blue',
    arrows=True
)

plt.show()
```

Vishal Acharya



145 You have been hired as a network analyst by a company to analyze the social network of their employees. The company has provided you with the following data:

There are 5 employees in the company, each identified by a unique ID from 1 to 5. The following relationships exist between the employees:

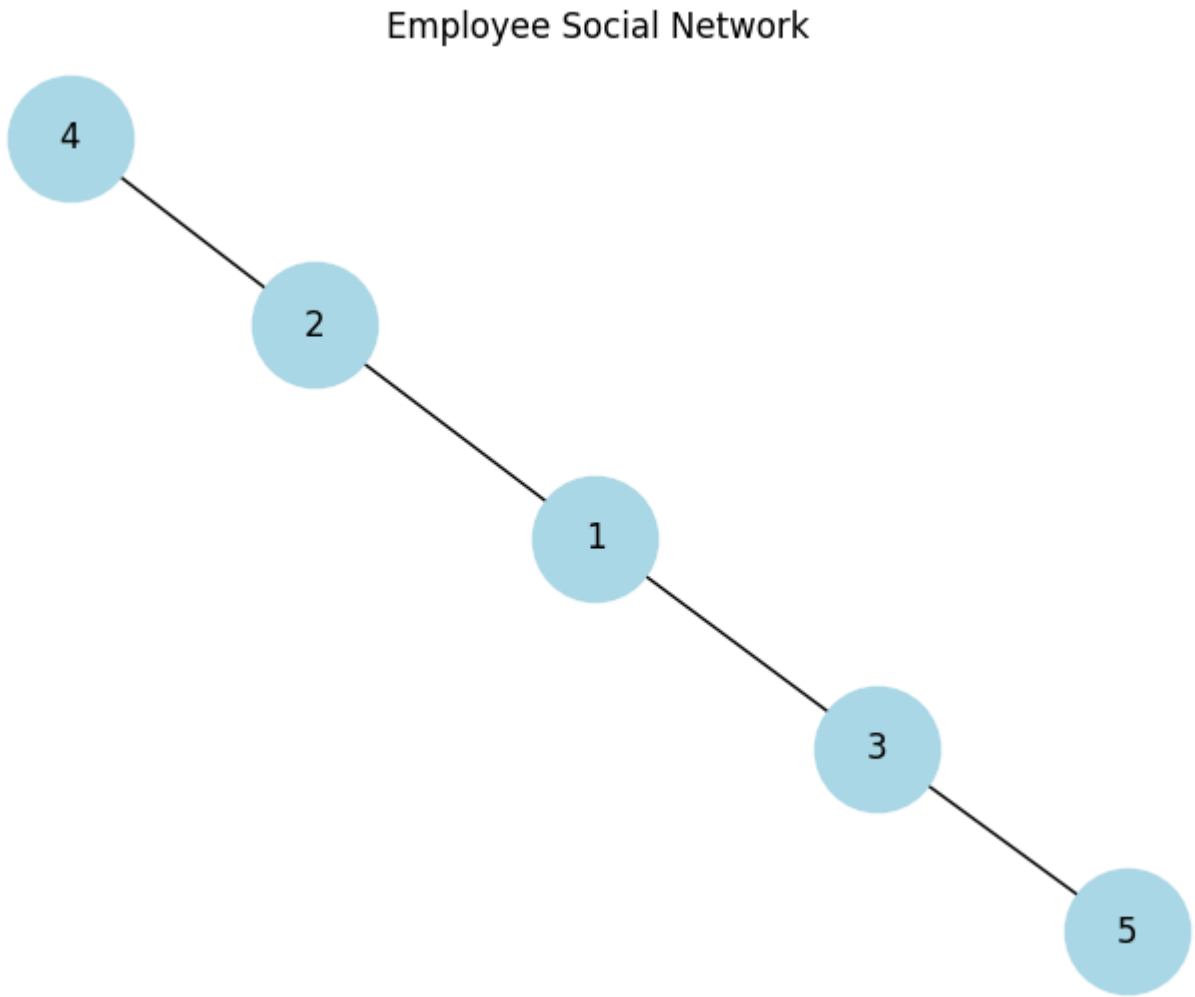
1. Employee 1 is friends with Employee 2 and Employee 3.
2. Employee 2 is friends with Employee 4.
3. Employee 3 is friends with Employee 5.

Your task is to create a NetworkX graph representing this social network and display it.

In [28]:
```
import networkx as nx
import matplotlib.pyplot as plt

# Create graph
G = nx.Graph()

# Add employees as nodes
G.add_nodes_from([1,2,3,4,5])

# Add friendship relationships (edges)
edges = [
    (1,2),
    (1,3),
    (2,4),
    (3,5)
]

G.add_edges_from(edges)

# Draw graph
nx.draw(G, with_labels=True, node_color='lightblue', node_size=2000)

plt.title("Employee Social Network")
plt.show()
```

Vishal Acharya

Vishal Acharya



146 "You have been hired by an Airlines company to analyze their routes. The company has provided you following data.

Your task is to create a NetworkX directed graph representing the routes and display it. Figure size should be (15,15), node color should be green, take appropriate node size, edge color should be red.

Data:

Vishal Acharya

- Kolkata to Mumbai
- Mumbai to Pune
- Mumbai to Goa
- Kolkata to Delhi
- Kolkata to Bhubaneshwar
- Mumbai to Delhi
- Delhi to Chandigarh
- Delhi to Surat
- Kolkata to Hyderabad
- Hyderabad to Chennai
- Chennai to Thiruvananthapuram
- Thiruvananthapuram to Hyderabad
- Kolkata to Varanasi
- Delhi to Varanasi
- Mumbai to Bangalore
- Chennai to Bangalore
- Hyderabad to Bangalore
- Kolkata to Guwahati "

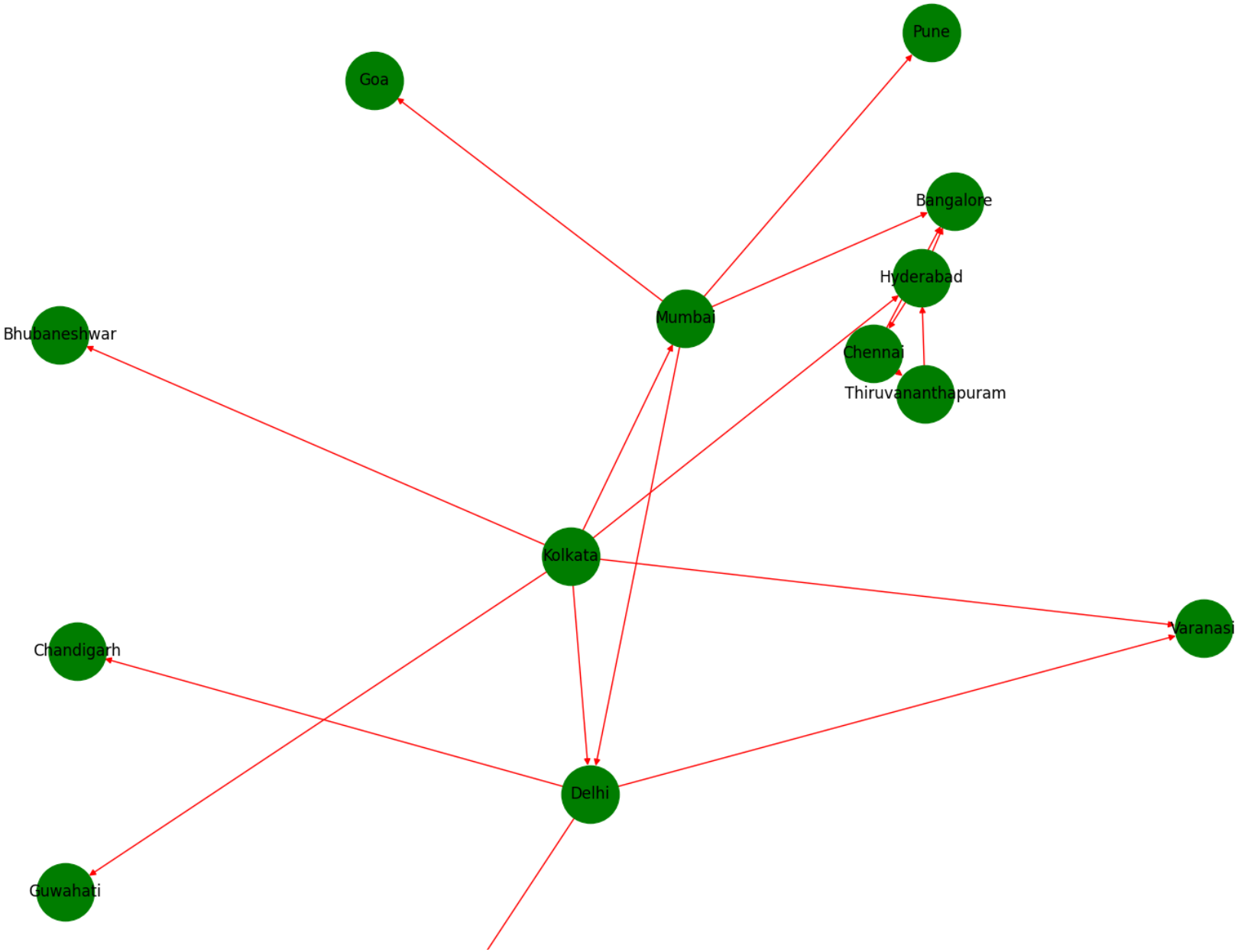
In [31]:
```
import networkx as nx
import matplotlib.pyplot as plt

# Create directed graph
G = nx.DiGraph()

# Airline routes
routes = [
("Kolkata","Mumbai"),
("Mumbai","Pune"),
("Mumbai","Goa"),
("Kolkata","Delhi"),
("Kolkata","Bhubaneshwar"),
("Mumbai","Delhi"),
("Delhi","Chandigarh"),
```

Vishal Acharya

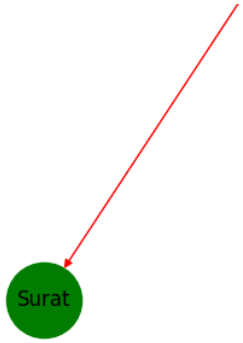
```
("Delhi","Surat"),
("Kolkata","Hyderabad"),
("Hyderabad","Chennai"),
("Chennai","Thiruvananthapuram"),
("Thiruvananthapuram","Hyderabad"),
("Kolkata","Varanasi"),
("Delhi","Varanasi"),
("Mumbai","Bangalore"),
("Chennai","Bangalore"),
("Hyderabad","Bangalore"),
("Kolkata","Guwahati")
]

# Add edges
G.add_edges_from(routes)

# Set figure size
plt.figure(figsize=(15,15))

nx.draw(G,
        with_labels=True,
        node_color="green",
        node_size=2000,
        edge_color="red",
        arrows=True)

plt.title("Airline Route Network")
plt.show()
```

Vishal Acharya

Airline Route Network



Vishal Acharya



147 "Using ‘supermarket_sales.csv’ file do the following operations and give required answer by using proper programming process.

1. Load the dataset into a pandas DataFrame and read first 8 rows.
2. Check for missing values and fill it by mean values of that particular column if any.
3. Find the number of orders which have ‘Quantity’ less than 3 and which have (either ‘Rating’ greater than 8.5 or ‘Total’ greater than 600).
4. Find the sum of ‘Total’ purchasing price spent by Member and Normal 'Customer type'.
5. Find the percentage of total of ‘gross income’ based on the different ‘Payment’ methods used by customers. (Ewallet, Cash and Credit card)
6. Analyze the purchasing behavior of male and female customers using ‘Gender’ column. Find their average purchase prices using ‘Total’ column.
7. Create a scatter plot that shows the relationship between total amount spent and rating.
8. Create a box plot that shows the distribution of ‘Rating’ and ‘Quantity’. And comment about outliers in both columns.
9. Visualize with parallel co-ordinates for ‘Unit price’, ‘Total’, ‘cogs’ columns’ data with respect to ‘Product line’."

Load dataset and display first 8 rows

In [33]:
```
import pandas as pd

df = pd.read_csv("supermarket_sales.csv")
```

```python
# first 8 rows
print(df.head(8))
```

Vishal Acharya

Vishal Acharya

```
    Invoice ID Branch       City Customer type  Gender  \
0  750-67-8428      A     Yangon        Member  Female
1  226-31-3081      C  Naypyitaw        Normal  Female
2  631-41-3108      A     Yangon        Normal    Male
3  123-19-1176      A     Yangon        Member    Male
4  373-73-7910      A     Yangon        Normal    Male
5  699-14-3026      C  Naypyitaw        Normal    Male
6  355-53-5943      A     Yangon        Member  Female
7  315-22-5665      C  Naypyitaw        Normal  Female

             Product line  Unit price  Quantity   Tax 5%     Total  \
0       Health and beauty       74.69         7  26.1415  548.9715
1  Electronic accessories       15.28         5   3.8200   80.2200
2      Home and lifestyle       46.33         7  16.2155  340.5255
3       Health and beauty       58.22         8  23.2880  489.0480
4       Sports and travel       86.31         7  30.2085  634.3785
5  Electronic accessories       85.39         7  29.8865  627.6165
6  Electronic accessories       68.84         6  20.6520  433.6920
7      Home and lifestyle       73.56        10  36.7800  772.3800

         Date   Time      Payment    cogs  gross margin percentage  \
0  01-05-2019  13:08      Ewallet  522.83                 4.761905
1  03-08-2019  10:29         Cash   76.40                 4.761905
2  03-03-2019  13:23  Credit card  324.31                 4.761905
3   1/27/2019  20:33      Ewallet  465.76                 4.761905
4  02-08-2019  10:37      Ewallet  604.17                 4.761905
5   3/25/2019  18:30      Ewallet  597.73                 4.761905
6   2/25/2019  14:36      Ewallet  413.04                 4.761905
7   2/24/2019  11:38      Ewallet  735.60                 4.761905

   gross income  Rating
0       26.1415     9.1
1        3.8200     9.6
2       16.2155     7.4
3       23.2880     8.4
4       30.2085     5.3
5       29.8865     4.1
6       20.6520     5.8
7       36.7800     8.0
```

Check missing values and fill with column mean

Vishal Acharya

In [34]:
```
# Check missing values
print(df.isnull().sum())

# Fill numeric missing values with mean
df.fillna(df.mean(numeric_only=True), inplace=True)
```

```
Invoice ID                 0
Branch                     0
City                       0
Customer type              0
Gender                     0
Product line               0
Unit price                 0
Quantity                   0
Tax 5%                     0
Total                      0
Date                       0
Time                       0
Payment                    0
cogs                       4
gross margin percentage    0
gross income               0
Rating                     5
dtype: int64
```

Number of orders with Quantity < 3 and (Rating > 8.5 OR Total > 600)

In [35]:
```
result = df[(df["Quantity"] < 3) & ((df["Rating"] > 8.5) | (df["Total"] > 600))]

print("Number of orders:", result.shape[0])
```

```
Number of orders: 45
```

Sum of Total purchasing price by Customer type

In [36]:
```
total_purchase = df.groupby("Customer type")["Total"].sum()

print(total_purchase)
```

Vishal Acharya

```
Customer type
Member    164223.444
Normal    158743.305
Name: Total, dtype: float64
```

Percentage of total gross income by Payment method

In [37]:
```
gross = df.groupby("Payment")["gross income"].sum()

percentage = (gross / gross.sum()) * 100

print(percentage)
```

```
Payment
Cash           34.742453
Credit card    31.200448
Ewallet        34.057099
Name: gross income, dtype: float64
```

Average purchase price by Gender

In [38]:
```
avg_purchase = df.groupby("Gender")["Total"].mean()

print(avg_purchase)
```

```
Gender
Female    335.095659
Male      310.789226
Name: Total, dtype: float64
```

Scatter plot (Total vs Rating)

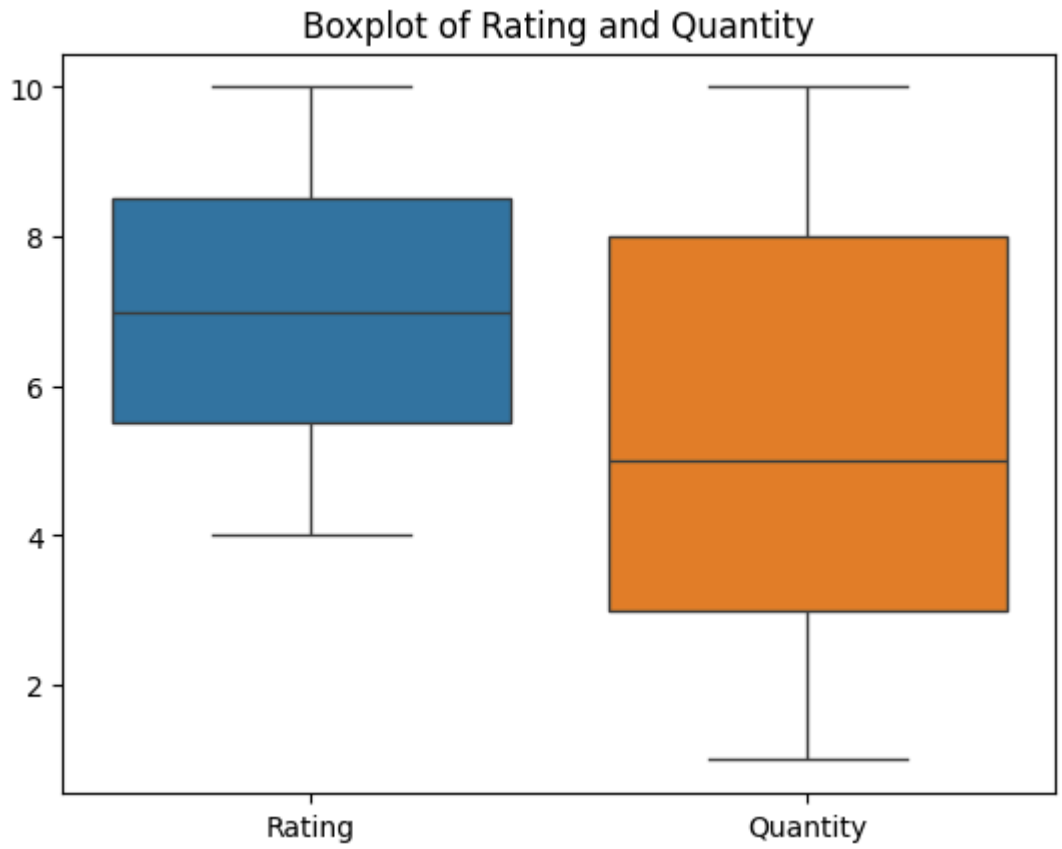
In [40]:
```
import seaborn as sns
import matplotlib.pyplot as plt

sns.scatterplot(x="Total", y="Rating", data=df)

plt.title("Total Amount Spent vs Rating")
plt.show()
```

Vishal Acharya

Boxplot for Rating and Quantity

In [41]:
```
sns.boxplot(data=df[["Rating","Quantity"]])

plt.title("Boxplot of Rating and Quantity")
plt.show()
```

Vishal Acharya

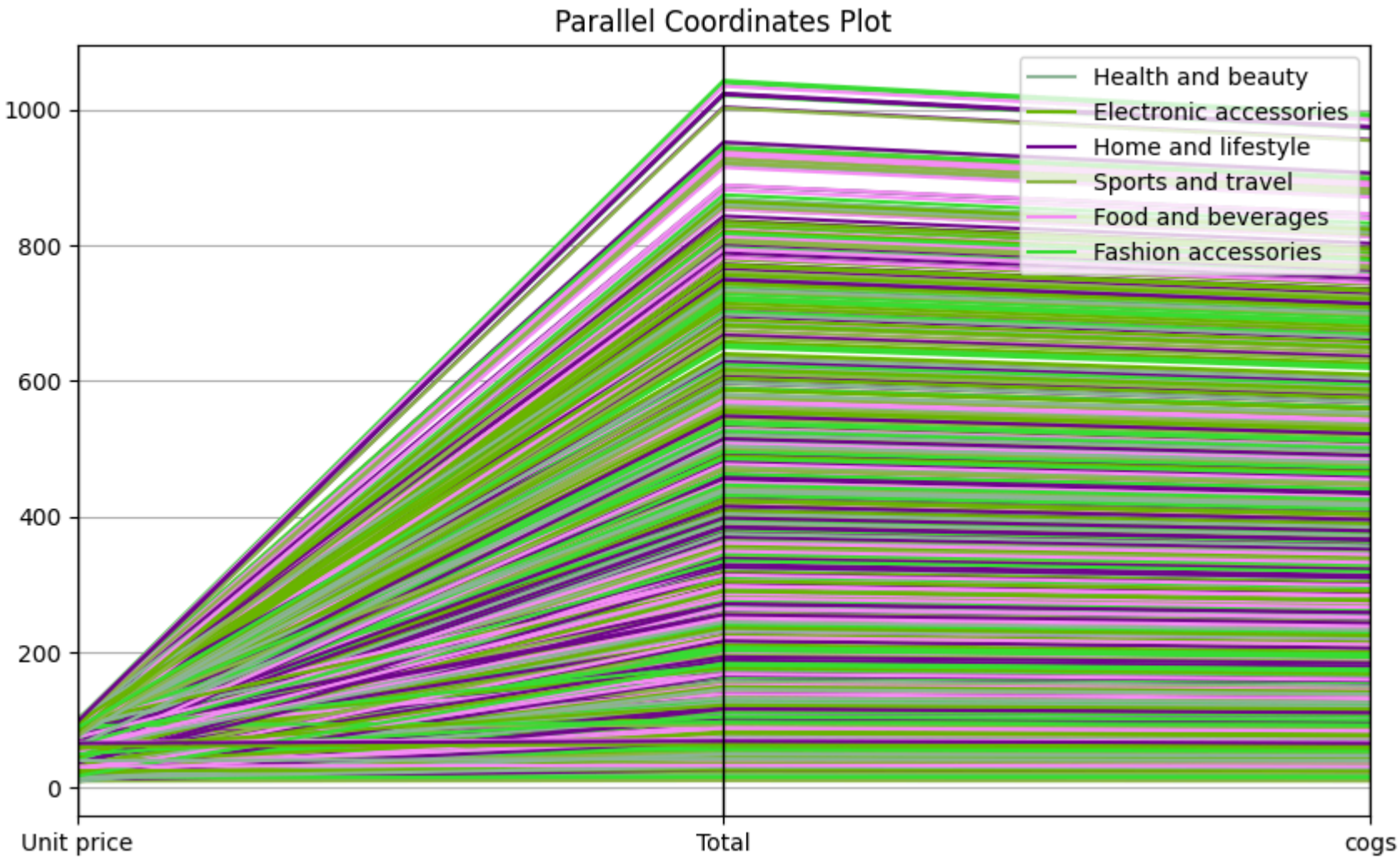


Parallel Coordinates Plot

Vishal Acharya

In [42]:
```
from pandas.plotting import parallel_coordinates

plt.figure(figsize=(10,6))

parallel_coordinates(
    df[["Product line","Unit price","Total","cogs"]],
    "Product line"
)

plt.title("Parallel Coordinates Plot")
plt.show()
```

Vishal Acharya



148 3Use the file data.csv which contains 169 rows and 4 columns.

1. Convert this file into pandas Data Frame and Display basic statistics like mean, std, quartiles, etc. for this data frame.
2. Create a correlation table for the data frame and comment about what kind of correlation is there between Duration and Calories?
3. Find whether there any null or NA values, drop all such rows if found in the data frame and print the shape of the data frame after dropping.

Vishal Acharya

4. Prepare a scatter matrix for the following data frame and prepare a parallel coordinates for Duration v/s Pulse, Maxpulse and Calories (all 3 other columns).
5. Do Maxpulse have any outliers? Find using function.
6. Show the outliers using box plot for Maxpulse, width of box plot should be 0.75 and notch should be True.
7. Create a scatter plot for Duration (x-axis) and then Pulse, Maxpulse and Calories (y-axis) with different colors. For each there should be different color and marker.

Convert CSV to DataFrame & Show Basic Statistics

In [48]:
```
import pandas as pd

# Load dataset
df = pd.read_csv("data (1).csv")

# Display basic statistics
print(df.describe())
```

```
         Duration       Pulse    Maxpulse     Calories
count  169.000000  169.000000  169.000000   164.000000
mean    63.846154  107.461538  134.047337   375.790244
std     42.299949   14.510259   16.450434   266.379919
min     15.000000   80.000000  100.000000    50.300000
25%     45.000000  100.000000  124.000000   250.925000
50%     60.000000  105.000000  131.000000   318.600000
75%     60.000000  111.000000  141.000000   387.600000
max    300.000000  159.000000  184.000000  1860.400000
```

Correlation Table

In [49]:
```
df.corr(numeric_only=True)
```

Vishal Acharya

Out[49]:

	Duration	Pulse	Maxpulse	Calories
Duration	1.000000	-0.155408	0.009403	0.922717
Pulse	-0.155408	1.000000	0.786535	0.025121
Maxpulse	0.009403	0.786535	1.000000	0.203813
Calories	0.922717	0.025121	0.203813	1.000000

Check Null Values and Drop Rows

In [50]:
```
# Check missing values
print(df.isnull().sum())

# Drop rows with NA values
df = df.dropna()

# Shape after dropping
print("Shape after dropping NA:", df.shape)
```

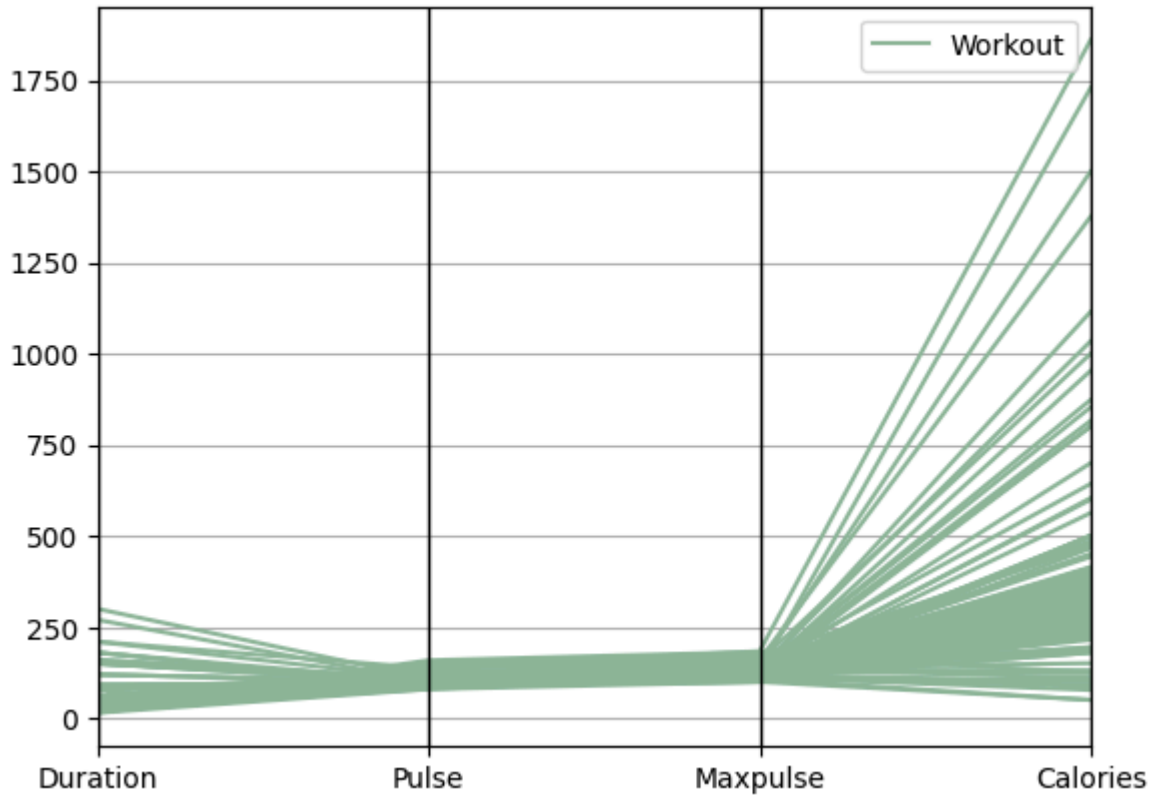
```
Duration    0
Pulse       0
Maxpulse    0
Calories    5
dtype: int64
Shape after dropping NA: (164, 4)
```

Parallel Coordinates

In [51]:
```
from pandas.plotting import parallel_coordinates

df_temp = df.copy()
df_temp["Category"] = "Workout"

parallel_coordinates(df_temp,
                     "Category",
                     cols=["Duration","Pulse","Maxpulse","Calories"])
```

Vishal Acharya

```
plt.show()
```



Detect Outliers in Maxpulse

In [52]:
```
Q1 = df["Maxpulse"].quantile(0.25)
Q3 = df["Maxpulse"].quantile(0.75)

IQR = Q3 - Q1

outliers = df[(df["Maxpulse"] < (Q1 - 1.5*IQR)) |
              (df["Maxpulse"] > (Q3 + 1.5*IQR))]

print(outliers)
```

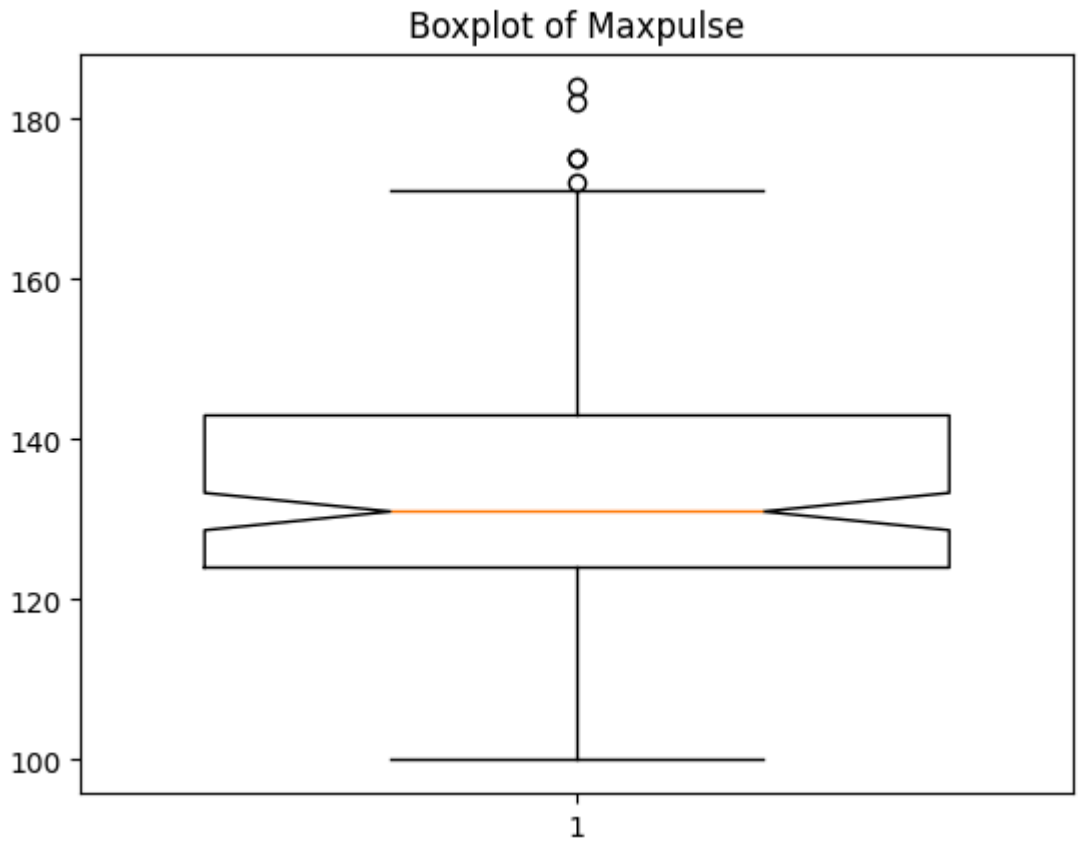
Vishal Acharya

```
     Duration  Pulse  Maxpulse  Calories
3          45    109       175     282.4
54         30    136       175     238.0
58         20    153       172     226.4
80         30    159       182     319.2
109       210    137       184    1860.4
```

Boxplot for Maxpulse

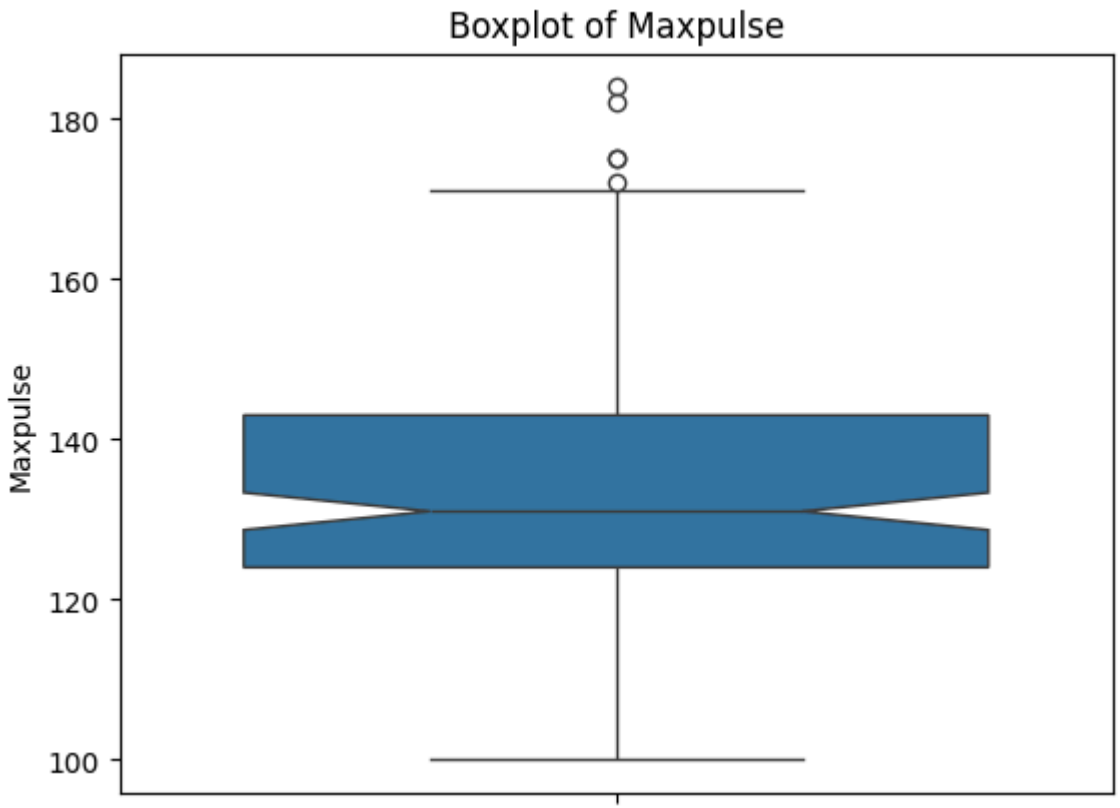
In [53]:
```python
import matplotlib.pyplot as plt

plt.boxplot(df["Maxpulse"], widths=0.75, notch=True)

plt.title("Boxplot of Maxpulse")
plt.show()
```

Vishal Acharya



In [54]:
```
sns.boxplot(y=df["Maxpulse"], width=0.75, notch=True)

plt.title("Boxplot of Maxpulse")
plt.show()
```

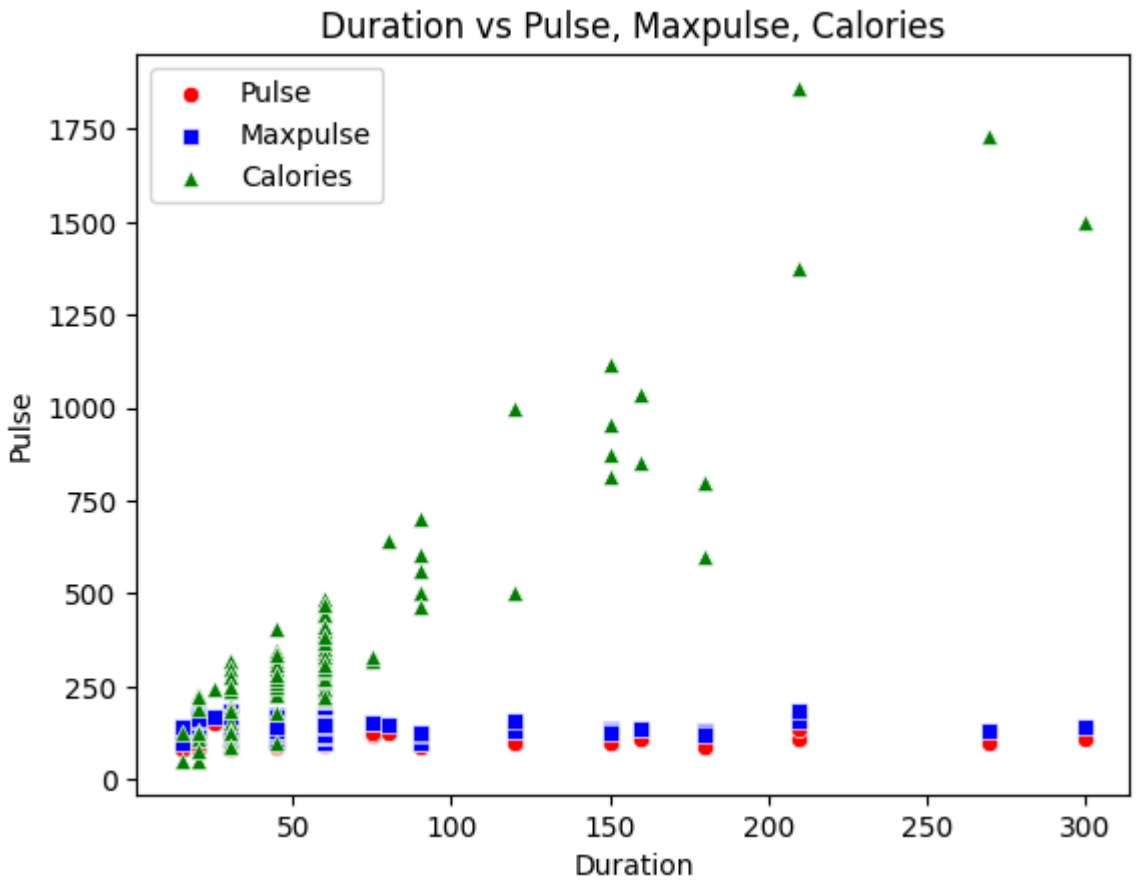
Vishal Acharya



Scatter Plot (Duration vs Pulse, Maxpulse, Calories)

In [55]:
```
sns.scatterplot(x="Duration", y="Pulse", data=df, color="red", marker="o", label="Pulse")
sns.scatterplot(x="Duration", y="Maxpulse", data=df, color="blue", marker="s", label="Maxpulse")
sns.scatterplot(x="Duration", y="Calories", data=df, color="green", marker="^", label="Calories")

plt.title("Duration vs Pulse, Maxpulse, Calories")
plt.legend()
plt.show()
```

Vishal Acharya



149 "For the given dataset – iris.csv, perform following exploratory data analysis using python -

Use comment feature to answer appropriate questions –

1. Load dataset into jupyter notebook using appropriate libraries. Check the datatypes of the dataset attributes. Does the data contain any missing /null values?
2. Extract head and tail of the dataset using appropriate methods.
3. Summarize statistical figures (i.e. mean, median, percentiles) in one table using appropriate method.
4. Create correlation table of all variables. What can you infer about relation between petal length and sepal length?
5. Create parallel coordinate plot of iris dataset. What can you infer about petal length and petal width?

Vishal Acharya

6. Create box plot of sepal width. Visualizing the plot, answer whether the sepal width data contains any outliers.
7. Create cross tabulation of sepal length and petal width attributes. What does the table represent?
8. Create scatter matrix of the dataset.
9. Create a new column called 'SepalLengthSize' which contains "High" if sepal length ≥ 5 or "Low" if sepal length < 5.

Load Dataset and Check Datatypes

In [58]:
```
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt
from pandas.plotting import parallel_coordinates, scatter_matrix

# Load dataset
df = pd.read_csv("iris.csv")

# Check datatypes
print(df.dtypes)

# Check for missing values
print(df.isnull().sum())

# Comment:
# If all values are 0 in isnull(), the dataset does not contain missing values.
```

```
Id                 int64
SepalLengthCm    float64
SepalWidthCm     float64
PetalLengthCm    float64
PetalWidthCm     float64
Species           object
dtype: object
Id               0
SepalLengthCm    0
SepalWidthCm     0
PetalLengthCm    0
PetalWidthCm     0
Species          0
dtype: int64
```

Vishal Acharya

Extract Head and Tail

In [59]:
```
# First 5 rows
print(df.head())

# Last 5 rows
print(df.tail())
```

```
   Id  SepalLengthCm  SepalWidthCm  PetalLengthCm  PetalWidthCm      Species
0   1            5.1           3.5            1.4           0.2  Iris-setosa
1   2            4.9           3.0            1.4           0.2  Iris-setosa
2   3            4.7           3.2            1.3           0.2  Iris-setosa
3   4            4.6           3.1            1.5           0.2  Iris-setosa
4   5            5.0           3.6            1.4           0.2  Iris-setosa
     Id  SepalLengthCm  SepalWidthCm  PetalLengthCm  PetalWidthCm  \
95   96            5.7           3.0            4.2           1.2
96   97            5.7           2.9            4.2           1.3
97   98            6.2           2.9            4.3           1.3
98   99            5.1           2.5            3.0           1.1
99  100            5.7           2.8            4.1           1.3

            Species
95  Iris-versicolor
96  Iris-versicolor
97  Iris-versicolor
98  Iris-versicolor
99  Iris-versicolor
```

Statistical Summary

In [60]:
```
stats = df.describe()
print(stats)

# Comment:
# describe() provides mean, std, min, max, and quartiles (25%, 50%, 75%).
# Median is represented by the 50% percentile.
```

Vishal Acharya

```
               Id  SepalLengthCm  SepalWidthCm  PetalLengthCm  PetalWidthCm
count  100.000000     100.000000    100.000000     100.000000    100.000000
mean    50.500000       5.471000      3.094000       2.862000      0.785000
std     29.011492       0.641698      0.476057       1.448565      0.566288
min      1.000000       4.300000      2.000000       1.000000      0.100000
25%     25.750000       5.000000      2.800000       1.500000      0.200000
50%     50.500000       5.400000      3.050000       2.450000      0.800000
75%     75.250000       5.900000      3.400000       4.325000      1.300000
max    100.000000       7.000000      4.400000       5.100000      1.800000
```

Correlation Table

In [61]:
```python
corr = df.corr(numeric_only=True)
print(corr)

# Comment:
# Petal Length and Sepal Length usually show a strong positive correlation.
# This means flowers with longer sepals tend to have longer petals.
```

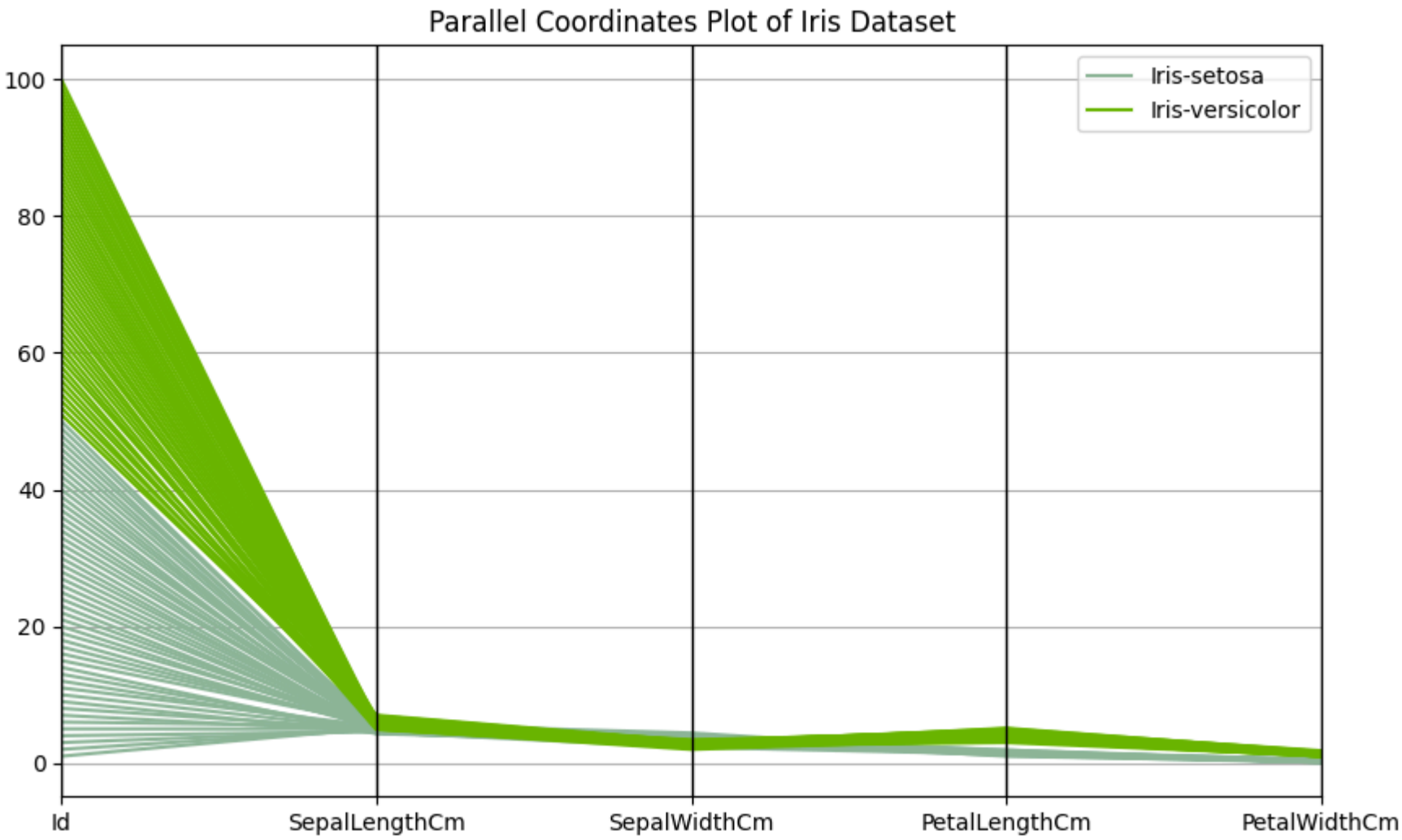
```
                     Id  SepalLengthCm  SepalWidthCm  PetalLengthCm  \
Id             1.000000       0.572341     -0.619103       0.826348
SepalLengthCm  0.572341       1.000000     -0.197646       0.812389
SepalWidthCm  -0.619103      -0.197646      1.000000      -0.595761
PetalLengthCm  0.826348       0.812389     -0.595761       1.000000
PetalWidthCm   0.821018       0.789613     -0.560120       0.978856

               PetalWidthCm
Id                 0.821018
SepalLengthCm      0.789613
SepalWidthCm      -0.560120
PetalLengthCm      0.978856
PetalWidthCm       1.000000
```

Parallel Coordinates Plot

In [64]:
```python
import pandas as pd
import matplotlib.pyplot as plt
from pandas.plotting import parallel_coordinates

# Load dataset
```

Vishal Acharya

```
df = pd.read_csv("iris.csv")

# Create parallel coordinates plot
plt.figure(figsize=(10,6))
parallel_coordinates(df, "Species")

plt.title("Parallel Coordinates Plot of Iris Dataset")
plt.show()
```



Vishal Acharya

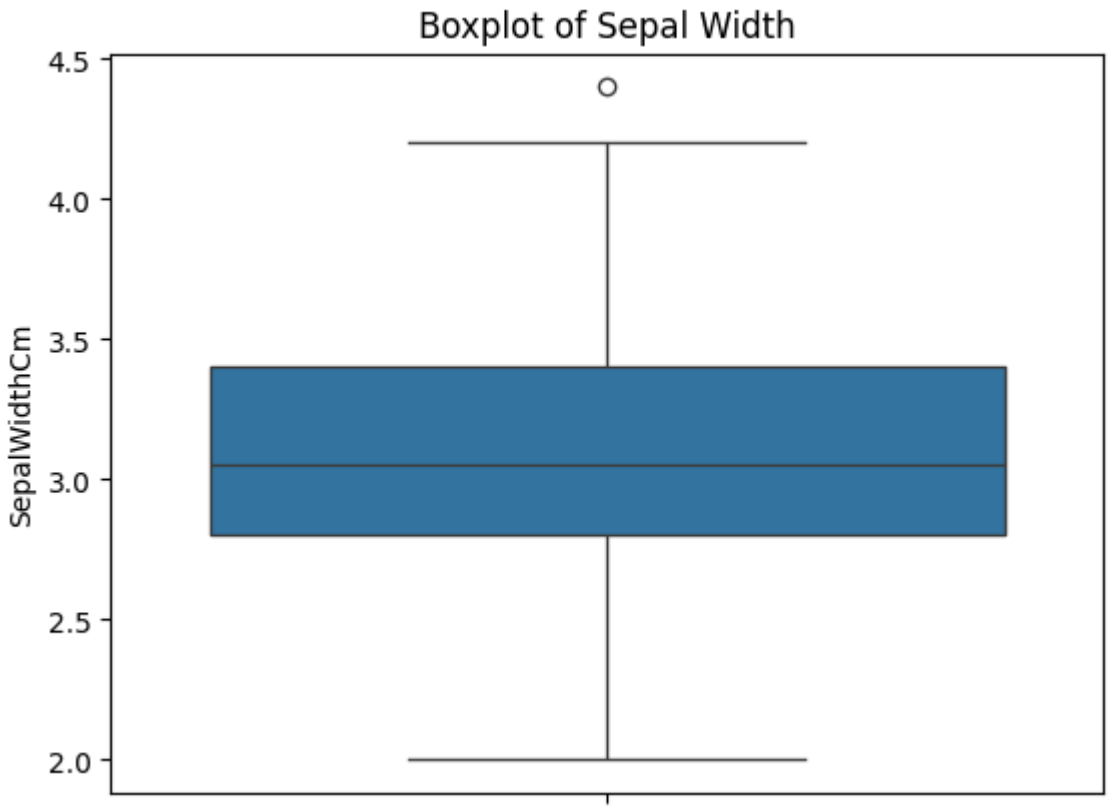
In [66]:
```
df.columns
```

Out[66]:
```
Index(['Id', 'SepalLengthCm', 'SepalWidthCm', 'PetalLengthCm', 'PetalWidthCm',
       'Species'],
      dtype='object')
```

Boxplot for Sepal Width

In [67]:
```
sns.boxplot(y=df['SepalWidthCm'])

plt.title("Boxplot of Sepal Width")
plt.show()

# Comment:
# Points outside whiskers represent outliers.
# Sepal width usually contains a few mild outliers.
```

Vishal Acharya



Cross Tabulation

In [70]:
```
crosstab = pd.crosstab(df["SepalLengthCm"], df['PetalWidthCm'])

print(crosstab)

# Comment:
# Crosstab shows the frequency distribution between
# sepal length and petal width values.
```

Vishal Acharya

PetalWidthCm	0.1	0.2	0.3	0.4	0.5	0.6	1.0	1.1	1.2	1.3	1.4	1.5 \
SepalLengthCm												
4.3	1	0	0	0	0	0	0	0	0	0	0	0
4.4	0	3	0	0	0	0	0	0	0	0	0	0
4.5	0	0	1	0	0	0	0	0	0	0	0	0
4.6	0	3	1	0	0	0	0	0	0	0	0	0
4.7	0	2	0	0	0	0	0	0	0	0	0	0
4.8	1	3	1	0	0	0	0	0	0	0	0	0
4.9	3	1	0	0	0	0	1	0	0	0	0	0
5.0	0	5	1	1	0	1	2	0	0	0	0	0
5.1	0	3	2	2	1	0	0	1	0	0	0	0
5.2	1	2	0	0	0	0	0	0	0	0	1	0
5.3	0	1	0	0	0	0	0	0	0	0	0	0
5.4	0	2	0	3	0	0	0	0	0	0	0	1
5.5	0	2	0	0	0	0	1	1	1	2	0	0
5.6	0	0	0	0	0	0	0	1	0	3	0	1
5.7	0	0	1	1	0	0	1	0	1	3	0	0
5.8	0	1	0	0	0	0	1	0	2	0	0	0
5.9	0	0	0	0	0	0	0	0	0	0	0	1
6.0	0	0	0	0	0	0	1	0	0	0	0	1
6.1	0	0	0	0	0	0	0	0	1	1	2	0
6.2	0	0	0	0	0	0	0	0	0	1	0	1
6.3	0	0	0	0	0	0	0	0	0	1	0	1
6.4	0	0	0	0	0	0	0	0	0	1	0	1
6.5	0	0	0	0	0	0	0	0	0	0	0	1
6.6	0	0	0	0	0	0	0	0	0	1	1	0
6.7	0	0	0	0	0	0	0	0	0	0	1	1
6.8	0	0	0	0	0	0	0	0	0	0	1	0
6.9	0	0	0	0	0	0	0	0	0	0	0	1
7.0	0	0	0	0	0	0	0	0	0	0	1	0

PetalWidthCm	1.6	1.7	1.8
SepalLengthCm			
4.3	0	0	0
4.4	0	0	0
4.5	0	0	0
4.6	0	0	0
4.7	0	0	0
4.8	0	0	0
4.9	0	0	0
5.0	0	0	0

Vishal Acharya

```
5.1              0    0    0
5.2              0    0    0
5.3              0    0    0
5.4              0    0    0
5.5              0    0    0
5.6              0    0    0
5.7              0    0    0
5.8              0    0    0
5.9              0    0    1
6.0              2    0    0
6.1              0    0    0
6.2              0    0    0
6.3              1    0    0
6.4              0    0    0
6.5              0    0    0
6.6              0    0    0
6.7              0    1    0
6.8              0    0    0
6.9              0    0    0
7.0              0    0    0
```

In [69]:
```
df.columns
```

Out[69]:
```
Index(['Id', 'SepalLengthCm', 'SepalWidthCm', 'PetalLengthCm', 'PetalWidthCm',
       'Species'],
      dtype='object')
```

Create New Column (SepalLengthSize)

In [73]:
```
df["SepalLengthSize"] = df["SepalLengthCm"].apply(
    lambda x: "High" if x >= 5 else "Low"
)

print(df.head())
```

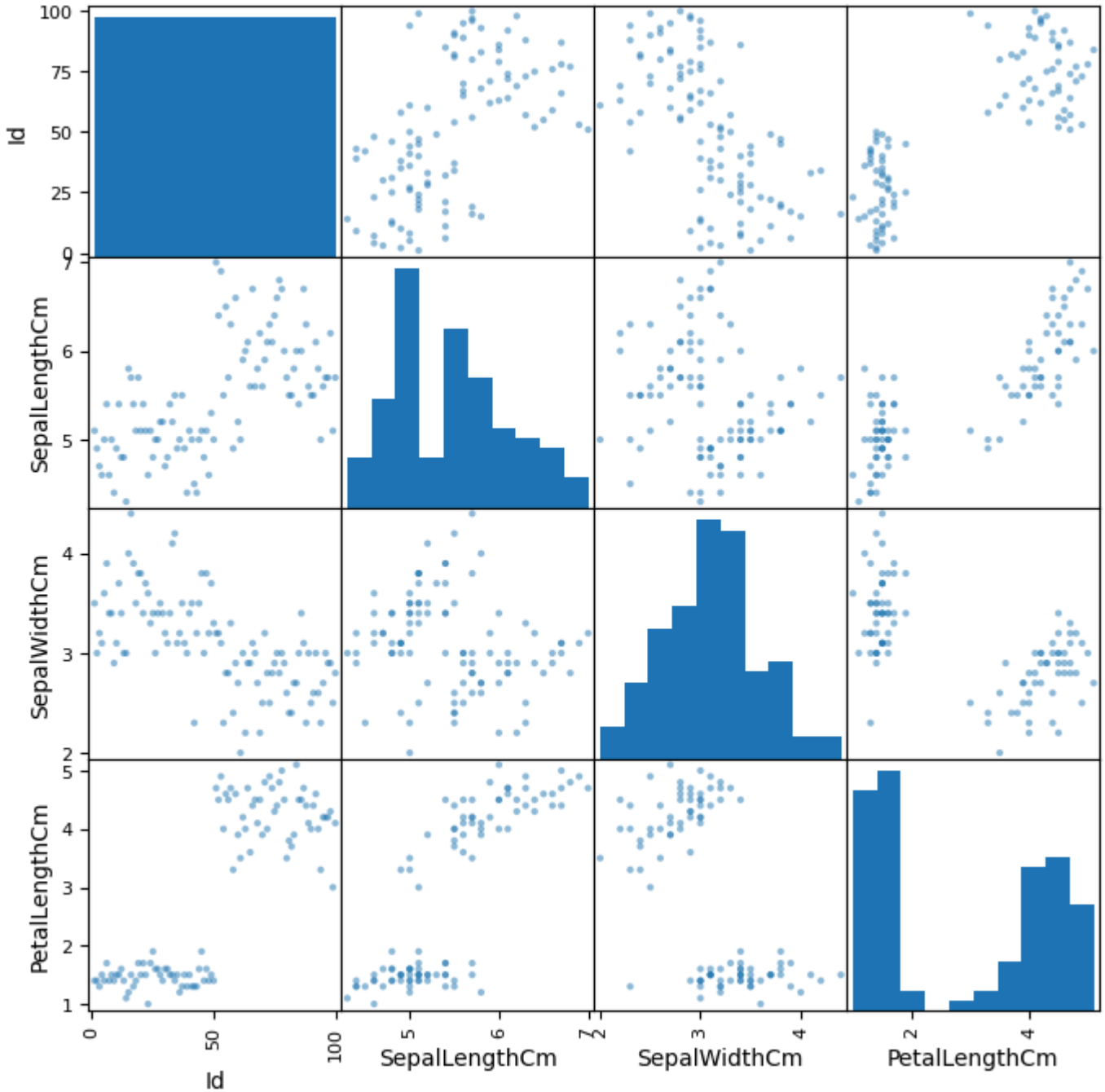
Vishal Acharya

```
   Id  SepalLengthCm  SepalWidthCm  PetalLengthCm  PetalWidthCm      Species  \
0   1            5.1           3.5            1.4           0.2  Iris-setosa
1   2            4.9           3.0            1.4           0.2  Iris-setosa
2   3            4.7           3.2            1.3           0.2  Iris-setosa
3   4            4.6           3.1            1.5           0.2  Iris-setosa
4   5            5.0           3.6            1.4           0.2  Iris-setosa

  SepalLengthSize
0            High
1             Low
2             Low
3             Low
4            High
```

Scatter Matrix

In [71]:
```
scatter_matrix(df.iloc[:,:4], figsize=(8,8))

plt.show()
```

Vishal Acharya



Vishal Acharya

150 "To upload the ‘diabetes_unclean.csv’ to your working folder

First import the following libraries import pandas as pd import matplotlib.pyplot as plt import numpy as np

1. Make a data frame with the variable name df
2. To display the specific statistics or measures that are relevant for object-type columns
3. To display the specific statistics or measures that are relevant for numerical-type columns
4. How many rows and columns are in a given dataset
5. To check the missing values
6. To replace the missing values in the column ""HbA1c"" with their mean value
7. Dropping the missing values of other columns
8. Display the correlation between variables
9. `Checking the outliers in the dataset for the following parameters: 'AGE', 'Urea', 'HbA1c', 'Chol', 'TG', 'HDL', 'LDL', 'VLDL', 'BMI' using box plot with labels and title`
10. Visualized the ""Urea"", ""HbA1c"", ""TG"" and ""BMI"" parameters for different ages using parallel_coordinates with labels and title
11. Remove the rows whose gender column has an “f” value and give the frequency count of the “F” and “M” values in different CLASS values
12. Remove the outliers in the ""HbA1c"" columns and print the shape of the data frame

Note: all task output with specific question numbers and follow the sequence Example: print(“Ans-1”)"

Make a data frame with the variable name df

In [77]:
```
df = pd.read_csv("diabetes_unclean.csv")
print("Ans-1")
print(df.head())
```

Vishal Acharya

```
Ans-1
    ID  No_Pation Gender   AGE  Urea    Cr  HbA1c  Chol   TG  HDL  LDL  VLDL  \
0  502      17975      F  50.0   4.7  46.0    4.9   4.2  0.9  2.4  1.4   0.5
1  735      34221      M  26.0   4.5  62.0    4.9   3.7  1.4  1.1  2.1   0.6
2  420      47975      F  50.0   4.7  46.0    4.9   4.2  0.9  2.4  1.4   0.5
3  680      87656      F  50.0   4.7  46.0    4.9   4.2  0.9  2.4  1.4   0.5
4  504      34223      M  33.0   7.1  46.0    4.9   4.9  1.0  0.8  2.0   0.4

    BMI CLASS
0  24.0     N
1  23.0     N
2  24.0     N
3  24.0     N
4  21.0     N
```

To display the specific statistics or measures that are relevant for object-type columns

In [78]:
```
print("\nAns-2")
print(df.describe(include=['object']))
```

```
Ans-2
       Gender CLASS
count    1009  1009
unique      3     5
top         M     Y
freq      570   840
```

To display the specific statistics or measures that are relevant for numerical-type columns

In [79]:
```
print("\nAns-3")
print(df.describe())
```

Vishal Acharya

```
Ans-3
                ID     No_Pation          AGE         Urea           Cr  \
count  1009.000000  1.009000e+03  1008.000000  1008.000000  1007.000000
mean    339.161546  2.717448e+05    53.620040     5.131094    68.973188
std     239.738169  3.365681e+06     8.740975     2.931136    59.813297
min       1.000000  1.230000e+02    25.000000     0.500000     6.000000
25%     127.000000  2.406500e+04    51.000000     3.700000    48.000000
50%     296.000000  3.439900e+04    55.000000     4.600000    60.000000
75%     548.000000  4.539000e+04    59.000000     5.700000    73.000000
max     800.000000  7.543566e+07    79.000000    38.900000   800.000000

             HbA1c         Chol           TG          HDL          LDL  \
count  1006.000000  1007.000000  1007.000000  1008.000000  1007.000000
mean      8.284155     4.863873     2.348769     1.204216     2.610119
std       2.533576     1.297326     1.397487     0.658158     1.116095
min       0.900000     0.000000     0.300000     0.200000     0.300000
25%       6.500000     4.000000     1.500000     0.900000     1.800000
50%       8.000000     4.800000     2.000000     1.100000     2.500000
75%      10.200000     5.600000     2.900000     1.300000     3.300000
max      16.000000    10.300000    13.800000     9.900000     9.900000

              VLDL          BMI
count  1008.000000  1009.000000
mean      1.850595    29.589911
std       3.649859     4.946246
min       0.100000    19.000000
25%       0.700000    26.000000
50%       0.900000    30.000000
75%       1.500000    33.000000
max      35.000000    47.750000
```

How many rows and columns are in a given dataset

In [81]:
```python
# Ans-4: Number of rows and columns
print("\nAns-4")
print("Shape of dataset:", df.shape)
```

```
Ans-4
Shape of dataset: (997, 14)
```

To check the missing values

Vishal Acharya

In [83]:
```
# Ans-5: Check missing values
print("\nAns-5")
print(df.isnull().sum())
```

```
Ans-5
ID           0
No_Pation    0
Gender       0
AGE          0
Urea         0
Cr           0
HbA1c        0
Chol         0
TG           0
HDL          0
LDL          0
VLDL         0
BMI          0
CLASS        0
dtype: int64
```

To replace the missing values in the column ""HbA1c"" with their mean value

In [84]:
```
# Ans-6: Replace missing values in HbA1c with mean
print("\nAns-6")
df["HbA1c"].fillna(df["HbA1c"].mean(), inplace=True)
print("Missing values in HbA1c replaced with mean")
```

```
Ans-6
Missing values in HbA1c replaced with mean
```

Vishal Acharya

```
C:\Users\VISHAL\AppData\Local\Temp\ipykernel_15384\2708975402.py:3: FutureWarning: A value is trying to be set on a copy of a D
ataFrame or Series through chained assignment using an inplace method.
The behavior will change in pandas 3.0. This inplace method will never work because the intermediate object on which we are set
ting values always behaves as a copy.

For example, when doing 'df[col].method(value, inplace=True)', try using 'df.method({col: value}, inplace=True)' or df[col] = d
f[col].method(value) instead, to perform the operation inplace on the original object.


  df["HbA1c"].fillna(df["HbA1c"].mean(), inplace=True)
```

Dropping the missing values of other columns

In [85]:
```
# Ans-7: Drop missing values in other columns
print("\nAns-7")
df = df.dropna()
print("Shape after dropping NA:", df.shape)
```

```
Ans-7
Shape after dropping NA: (997, 14)
```

Display the correlation between variables

In [86]:
```
# Ans-8: Correlation between variables
print("\nAns-8")
corr = df.corr(numeric_only=True)
print(corr)

sns.heatmap(corr, annot=True, cmap="coolwarm")
plt.title("Correlation Matrix")
plt.show()
```

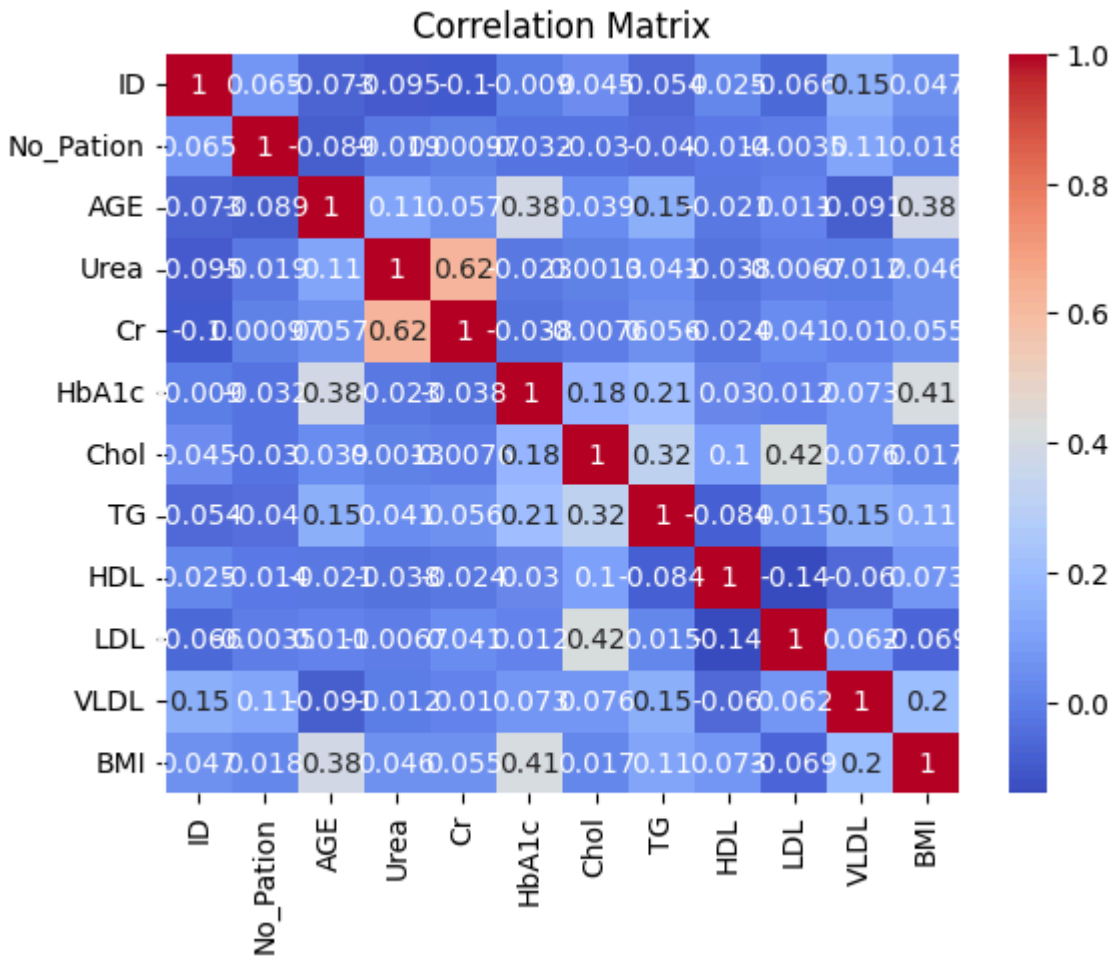
Vishal Acharya

Ans-8

	ID	No_Pation	AGE	Urea	Cr	HbA1c	\
ID	1.000000	0.064599	-0.072687	-0.094891	-0.100046	-0.009037	
No_Pation	0.064599	1.000000	-0.088870	-0.019061	0.000973	-0.032350	
AGE	-0.072687	-0.088870	1.000000	0.108613	0.056940	0.384675	
Urea	-0.094891	-0.019061	0.108613	1.000000	0.624810	-0.023307	
Cr	-0.100046	0.000973	0.056940	0.624810	1.000000	-0.037735	
HbA1c	-0.009037	-0.032350	0.384675	-0.023307	-0.037735	1.000000	
Chol	0.045414	-0.030288	0.038966	0.001286	-0.007636	0.177676	
TG	-0.054110	-0.039859	0.149274	0.040939	0.056031	0.214030	
HDL	0.025226	-0.013554	-0.021029	-0.037843	-0.023578	0.030455	
LDL	-0.065718	-0.003520	0.011496	-0.006673	0.040981	0.011536	
VLDL	0.145700	0.113635	-0.090796	-0.011573	0.010328	0.072641	
BMI	0.047270	0.017738	0.381176	0.045753	0.054847	0.413130	

	Chol	TG	HDL	LDL	VLDL	BMI
ID	0.045414	-0.054110	0.025226	-0.065718	0.145700	0.047270
No_Pation	-0.030288	-0.039859	-0.013554	-0.003520	0.113635	0.017738
AGE	0.038966	0.149274	-0.021029	0.011496	-0.090796	0.381176
Urea	0.001286	0.040939	-0.037843	-0.006673	-0.011573	0.045753
Cr	-0.007636	0.056031	-0.023578	0.040981	0.010328	0.054847
HbA1c	0.177676	0.214030	0.030455	0.011536	0.072641	0.413130
Chol	1.000000	0.318894	0.103370	0.419237	0.076373	0.016989
TG	0.318894	1.000000	-0.083548	0.015099	0.145825	0.110120
HDL	0.103370	-0.083548	1.000000	-0.141771	-0.059767	0.072581
LDL	0.419237	0.015099	-0.141771	1.000000	0.062445	-0.068723
VLDL	0.076373	0.145825	-0.059767	0.062445	1.000000	0.198048
BMI	0.016989	0.110120	0.072581	-0.068723	0.198048	1.000000

Vishal Acharya



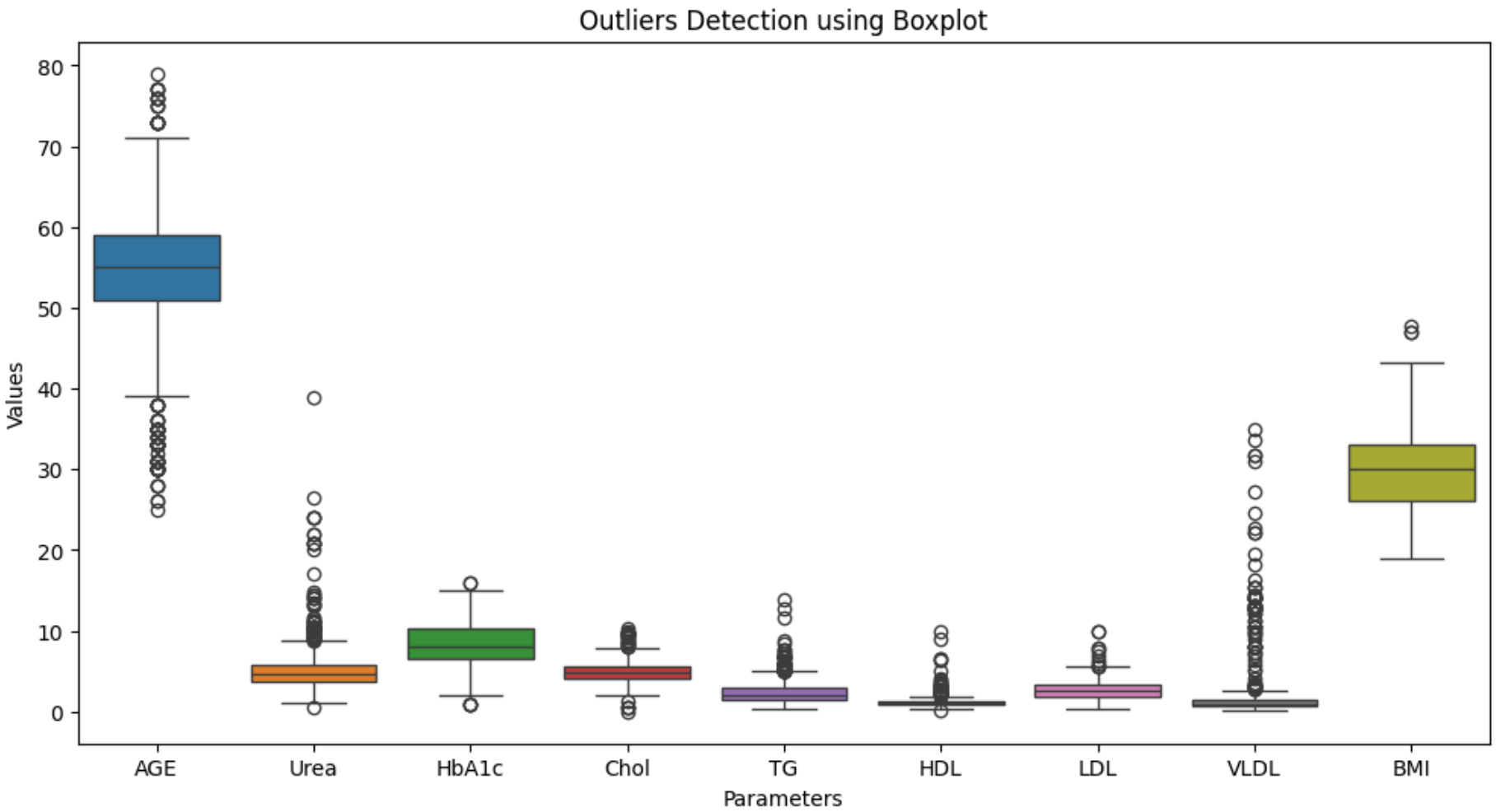
Checking the outliers in the dataset for the following parameters: 'AGE', 'Urea', 'HbA1c', 'Chol', 'TG', 'HDL', 'LDL', 'VLDL', 'BMI' using box plot with labels and title

In [87]:
```
# Ans-9: Check outliers using boxplot
print("\nAns-9")

cols = ['AGE','Urea','HbA1c','Chol','TG','HDL','LDL','VLDL','BMI']

plt.figure(figsize=(12,6))
sns.boxplot(data=df[cols])
```

Vishal Acharya

```
plt.title("Outliers Detection using Boxplot")
plt.xlabel("Parameters")
plt.ylabel("Values")
plt.show()
```

```
Ans-9
```



Visualized the ""Urea"", ""HbA1c"", ""TG"" and ""BMI"" parameters for different ages using parallel_coordinates with labels and title

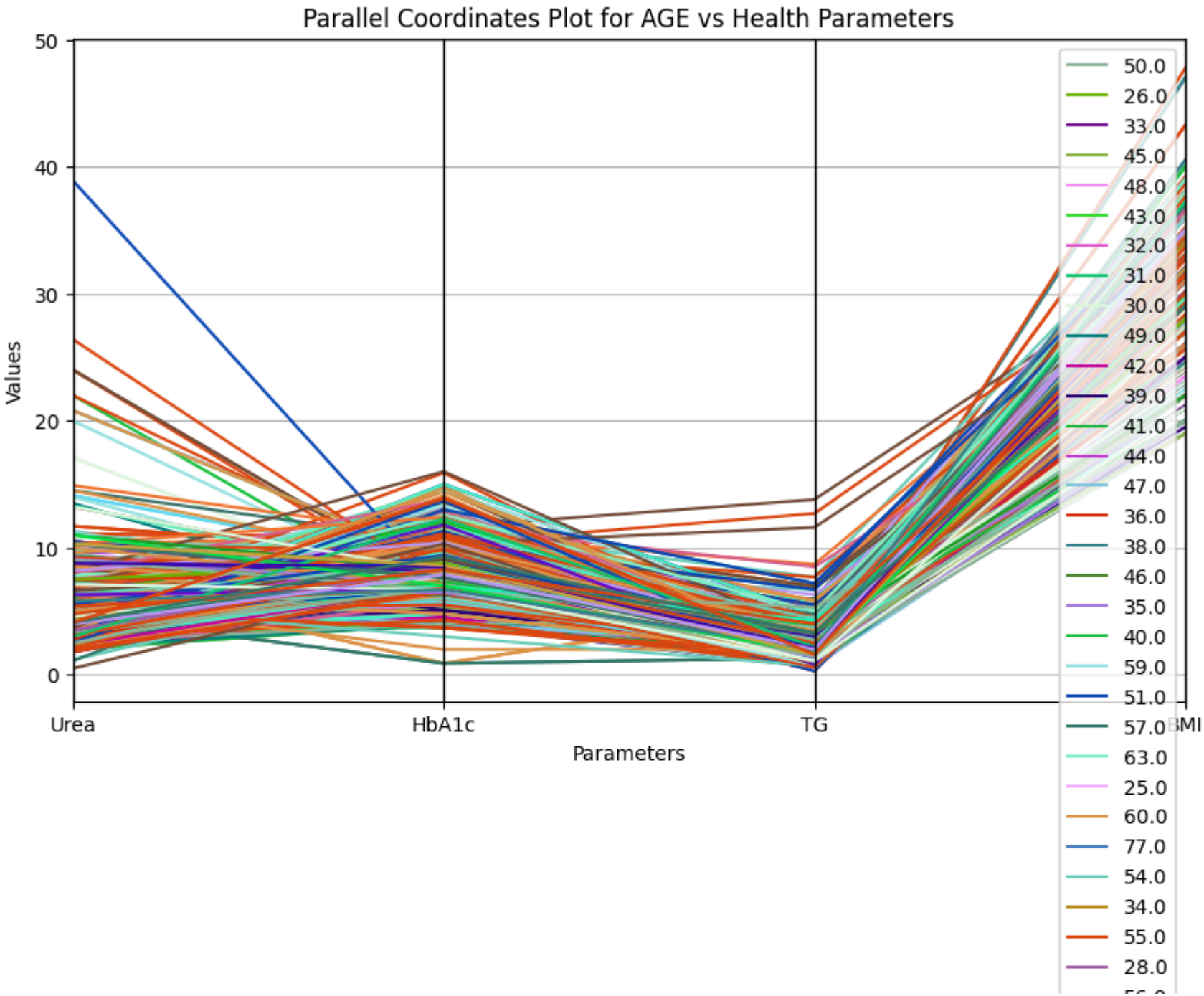
In [88]:
```
# Ans-10: Parallel Coordinates Plot
print("\nAns-10")
```

Vishal Acharya

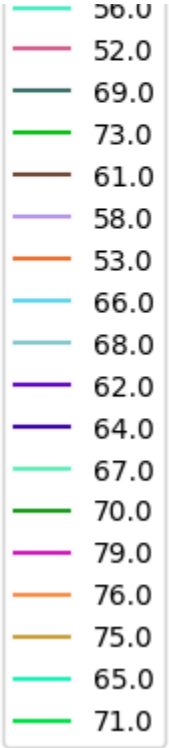
```
plt.figure(figsize=(10,6))
parallel_coordinates(df[['AGE','Urea','HbA1c','TG','BMI']], 'AGE')
plt.title("Parallel Coordinates Plot for AGE vs Health Parameters")
plt.xlabel("Parameters")
plt.ylabel("Values")
plt.show()
```

Ans-10

Vishal Acharya



Vishal Acharya



Remove the rows whose gender column has an “f” value and give the frequency count of the “F” and “M” values in different CLASS values

In [89]:
```
# Ans-11: Remove rows where gender = 'f' and frequency of F and M in CLASS
print("\nAns-11")

df2 = df[df["Gender"] != "f"]

freq = pd.crosstab(df2["CLASS"], df2["Gender"])
print(freq)
```

Vishal Acharya

```
Ans-11
Gender     F    M
CLASS
N         64   38
N          0    1
P         17   36
Y        349  482
Y          2    6
```

Remove the outliers in the ""HbA1c"" columns and print the shape of the data frame Note: all task output with specific question numbers and follow the sequence Example: print(“Ans-1”)"

In [90]:
```
# Ans-12: Remove outliers in HbA1c using IQR
print("\nAns-12")

Q1 = df["HbA1c"].quantile(0.25)
Q3 = df["HbA1c"].quantile(0.75)
IQR = Q3 - Q1

df_no_outliers = df[(df["HbA1c"] >= Q1 - 1.5*IQR) & (df["HbA1c"] <= Q3 + 1.5*IQR)]

print("Shape after removing HbA1c outliers:", df_no_outliers.shape)
```

```
Ans-12
Shape after removing HbA1c outliers: (991, 14)
```

151 "import pandas as pd

import seaborn as sns import matplotlib.pyplot as plt

df = pd.DataFrame({ ""Math"":[78,85,90,66,88], ""Science"":[80,82,89,70,85], ""English"":[75,88,92,60,84], ""Computer"":[90,91,95,72,89] }, index= [""S1"",""S2"",""S3"",""S4"",""S5""])

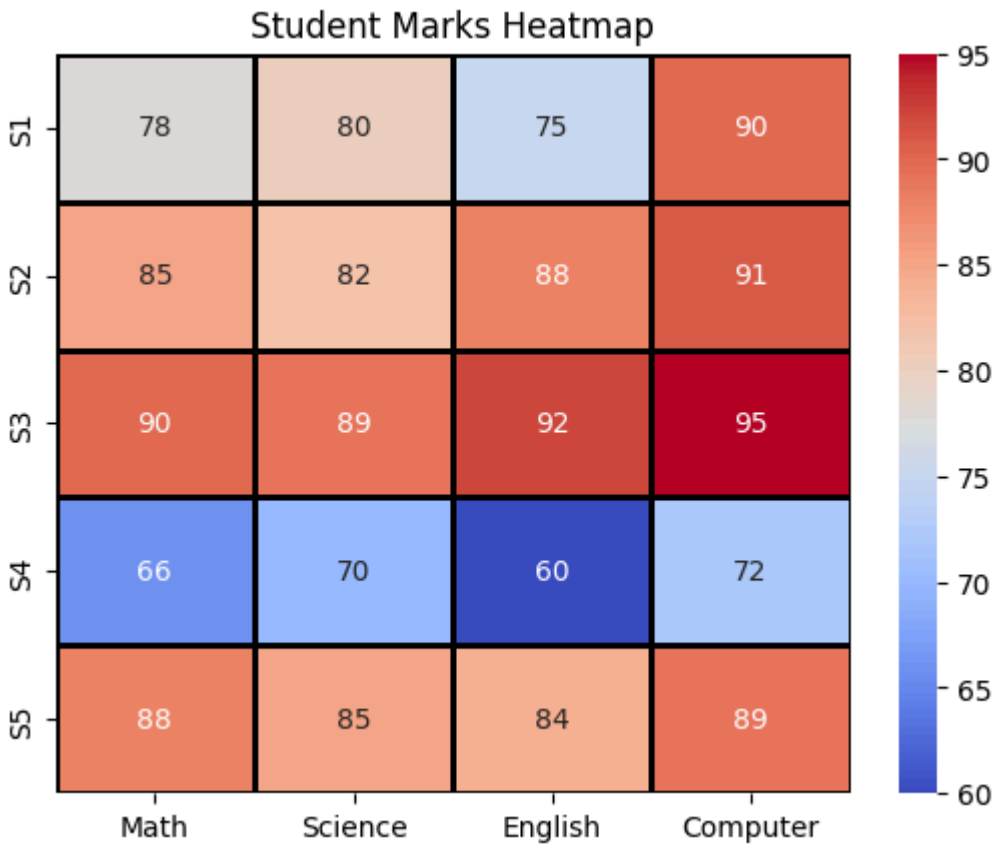
Plot heatmap,,annot=True,cmap=""coolwarm"", linewidths=1, linecolor=""black"" Show colorbar

In [1]:
```
import pandas as pd
import seaborn as sns
```

Vishal Acharya

```python
import matplotlib.pyplot as plt

# Create DataFrame
df = pd.DataFrame({
    "Math":[78,85,90,66,88],
    "Science":[80,82,89,70,85],
    "English":[75,88,92,60,84],
    "Computer":[90,91,95,72,89]
}, index=["S1","S2","S3","S4","S5"])

# Plot Heatmap
sns.heatmap(
    df,
    annot=True,              # show values
    cmap="coolwarm",         # color theme
    linewidths=1,            # grid line width
    linecolor="black",       # grid line color
    cbar=True                # show color bar
)

plt.title("Student Marks Heatmap")
plt.show()
```

Vishal Acharya



152 "import pandas as pd

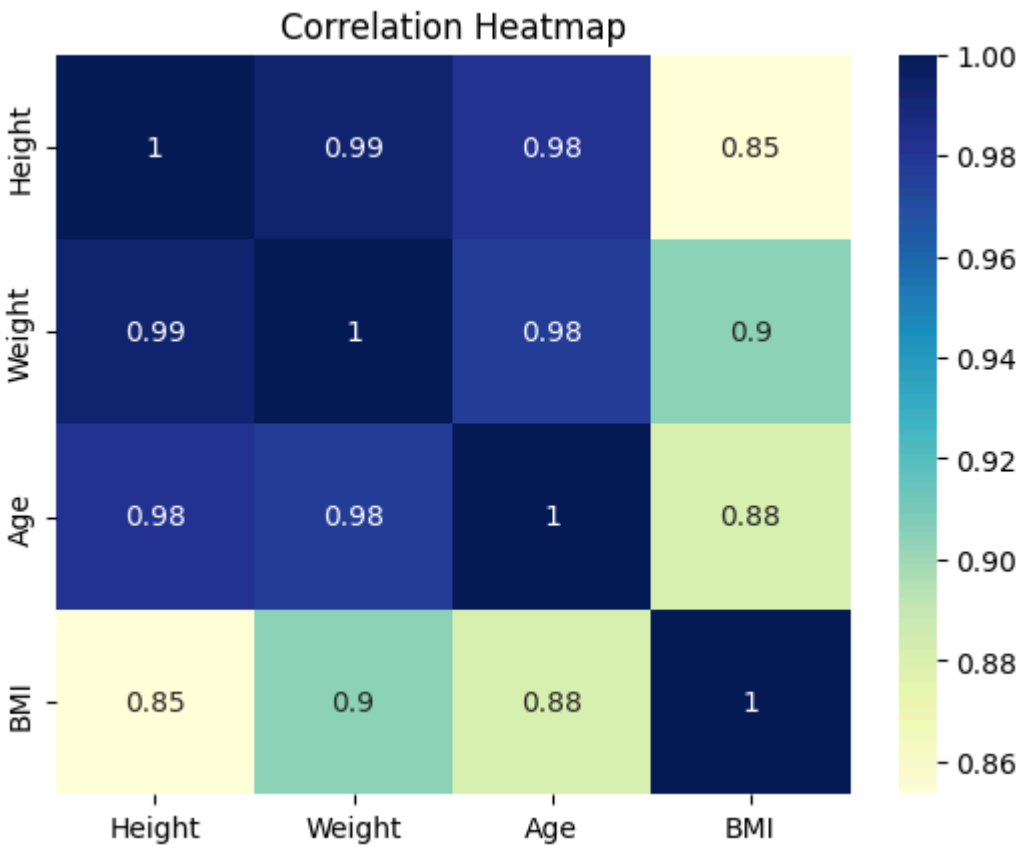
df = pd.DataFrame({ ""Height"":[150,160,170,180,175], ""Weight"":[50,60,65,75,70], ""Age"":[20,22,25,30,28], ""BMI"":[22,24,23,26,25] })

Compute correlation, Plot heatmap with cmap=""YlGnBu"" "

In [2]:
```
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt

# Create DataFrame + Compute correlation
```

Vishal Acharya

```
corr = pd.DataFrame({
    "Height":[150,160,170,180,175],
    "Weight":[50,60,65,75,70],
    "Age":[20,22,25,30,28],
    "BMI":[22,24,23,26,25]
}).corr()

# Plot heatmap
sns.heatmap(corr, cmap="YlGnBu", annot=True)
plt.title("Correlation Heatmap")
plt.show()
```



153 "import pandas as pd

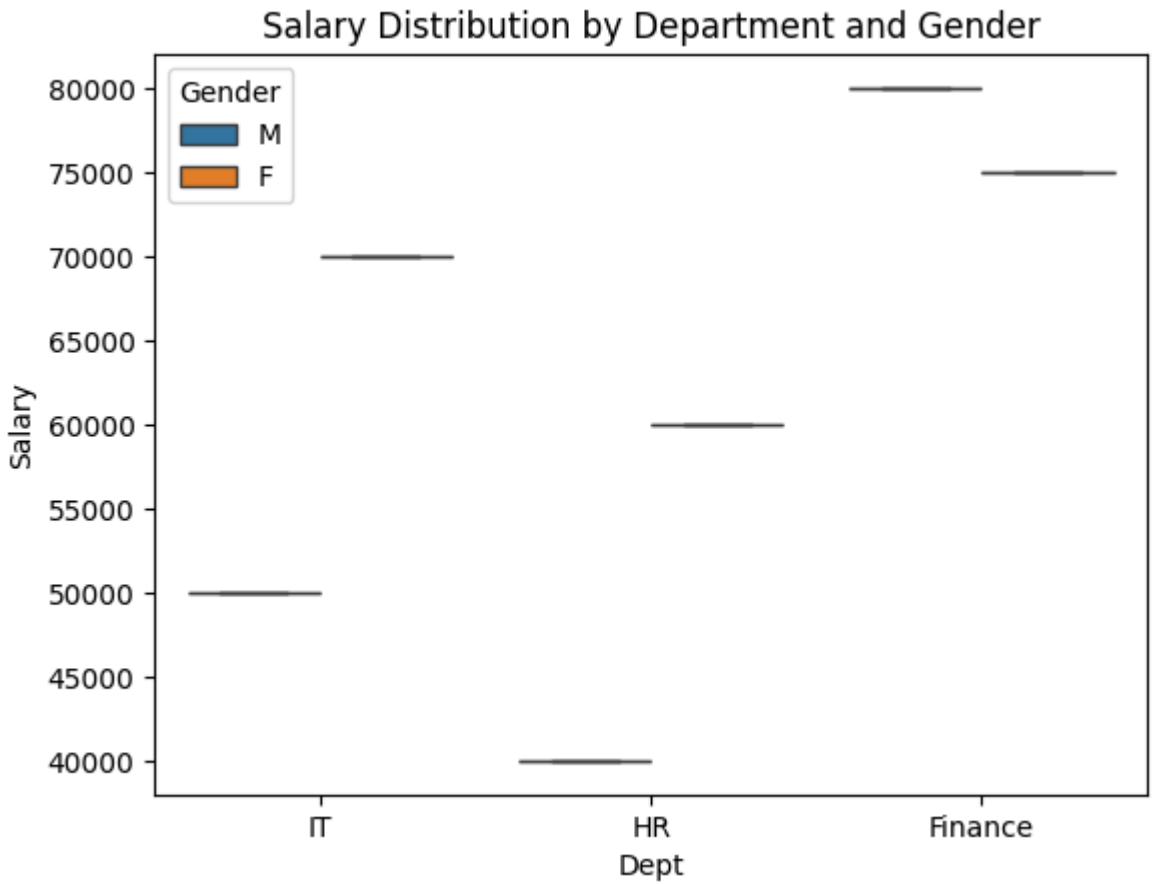
df = pd.DataFrame({ ""Dept"":[""IT"",""IT"",""HR"",""HR"",""Finance"",""Finance""], ""Salary"":[50000,70000,40000,60000,80000,75000], ""Gender"":[""M"",""F"",""M"",""F"",""M"",""F""] })

Plot boxplot of Salary vs Dept with hue=Gender. "

In [3]:
```
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt

# Create DataFrame
df = pd.DataFrame({
    "Dept":["IT","IT","HR","HR","Finance","Finance"],
    "Salary":[50000,70000,40000,60000,80000,75000],
    "Gender":["M","F","M","F","M","F"]
})

# Plot boxplot
sns.boxplot(x="Dept", y="Salary", hue="Gender", data=df)
plt.title("Salary Distribution by Department and Gender")
plt.show()
```

Vishal Acharya

Vishal Acharya



154 "import pandas as pd

df = pd.DataFrame({ ""Salary"":[20000,22000,21000,23000,25000,300000] })

Plot boxplot and observe outlier. "

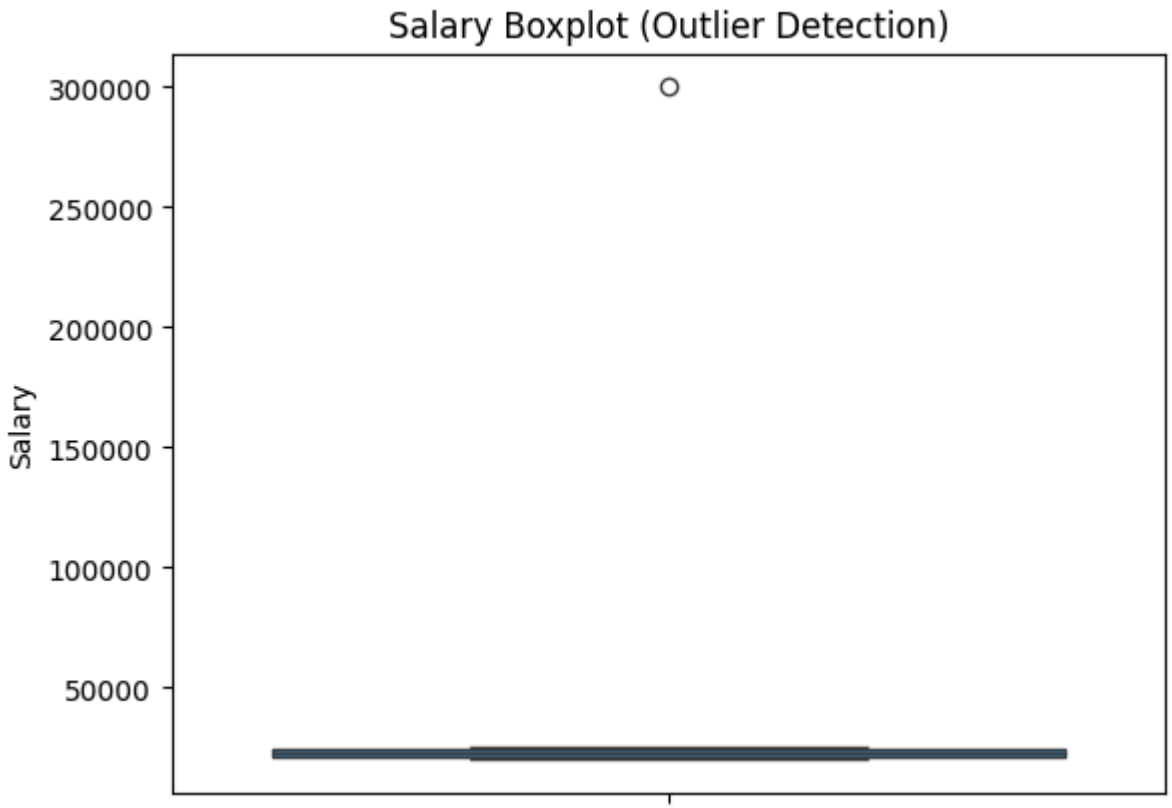
In [4]:
```
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt
```

Vishal Acharya

```python
# Create DataFrame
df = pd.DataFrame({
    "Salary":[20000,22000,21000,23000,25000,300000]
})

# -------- IQR Calculation --------
Q1 = df["Salary"].quantile(0.25)
Q3 = df["Salary"].quantile(0.75)
IQR = Q3 - Q1

lower_bound = Q1 - 1.5 * IQR
upper_bound = Q3 + 1.5 * IQR

# Detect outliers
outliers = df[(df["Salary"] < lower_bound) | (df["Salary"] > upper_bound)]

print("Q1:", Q1)
print("Q3:", Q3)
print("IQR:", IQR)
print("Lower Bound:", lower_bound)
print("Upper Bound:", upper_bound)
print("Outliers:\n", outliers)

# -------- Boxplot --------
sns.boxplot(y="Salary", data=df)
plt.title("Salary Boxplot (Outlier Detection)")
plt.show()
```

```
Q1: 21250.0
Q3: 24500.0
IQR: 3250.0
Lower Bound: 16375.0
Upper Bound: 29375.0
Outliers:
    Salary
5  300000
```

Vishal Acharya



155 "import pandas as pd

df = pd.DataFrame({ ""StudyHours"":[2,3,5,6,8], ""Marks"":[50,60,75,85,95], ""Gender"":[""M"",""F"",""M"",""F"",""M""] })

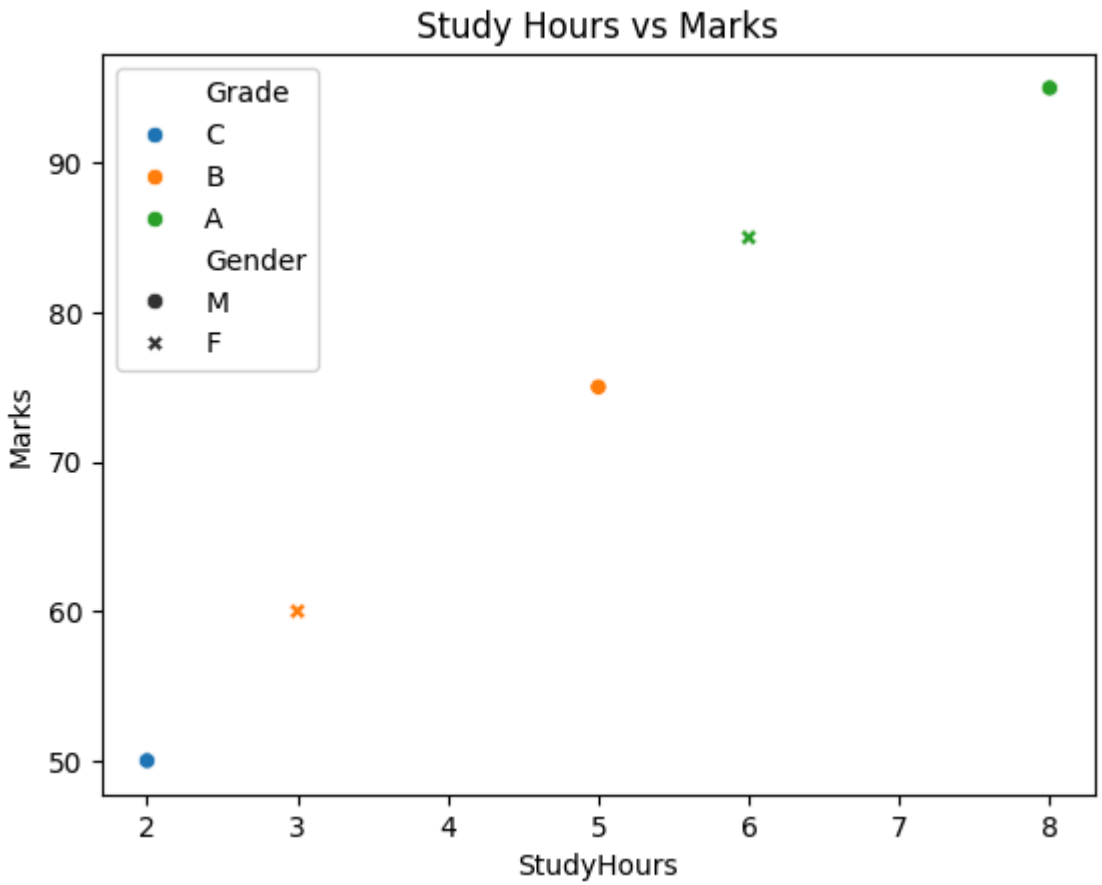
Create dataset of students with Marks and StudyHours. Use sns.scatterplot() Use hue for Grade. Use style for Gender. "

In [5]:
```
import pandas as pd
import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt

# Create DataFrame
```

Vishal Acharya

```python
df = pd.DataFrame({
    "StudyHours":[2,3,5,6,8],
    "Marks":[50,60,75,85,95],
    "Gender":["M","F","M","F","M"]
})

# Create Grade using np.where
df["Grade"] = np.where(df["Marks"] >= 80, "A",
                np.where(df["Marks"] >= 60, "B", "C"))

# Scatter Plot
sns.scatterplot(x="StudyHours", y="Marks",
                hue="Grade", style="Gender", data=df)

plt.title("Study Hours vs Marks")
plt.show()
```

Vishal Acharya



156 "Simple Bar Chart:Visualize total revenue of different departments.

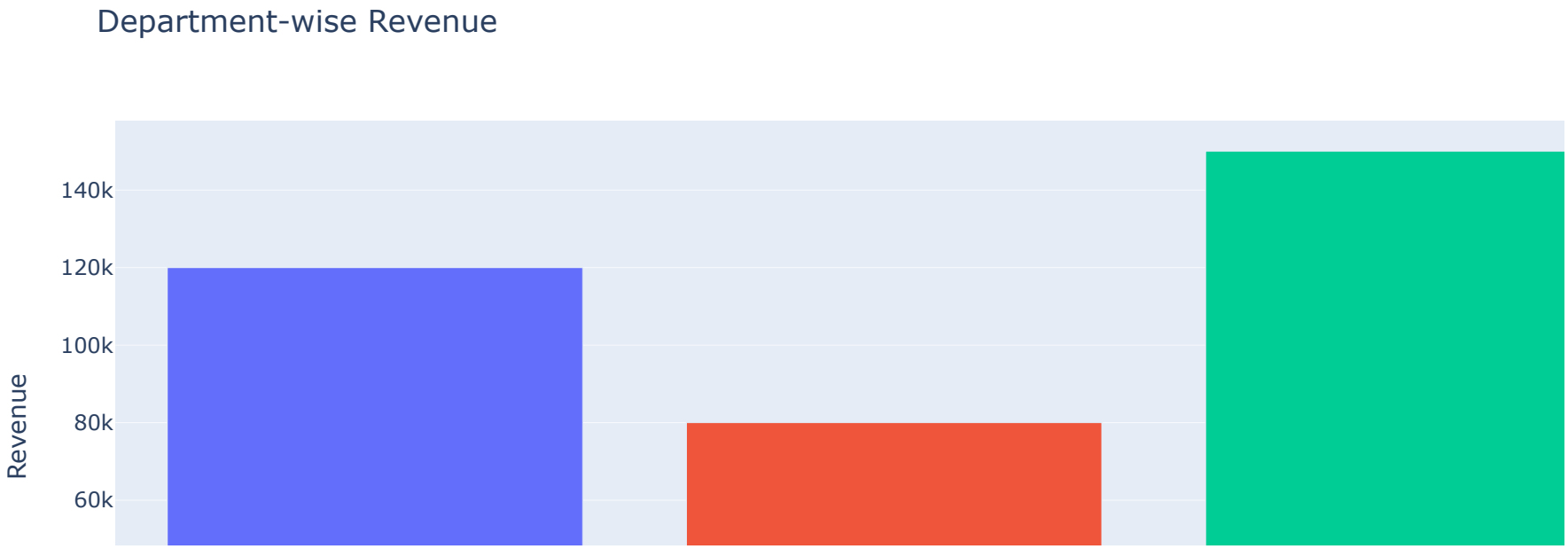
import pandas as pd import plotly.express as px import plotly.io as pio pio.renderers.default = ""notebook"" df = pd.DataFrame({ ""Department"": [""IT"", ""HR"", ""Finance"", ""Marketing""], ""Revenue"": [120000, 80000, 150000, 95000] }) "

In [6]:
```
import pandas as pd
import plotly.express as px
import plotly.io as pio

pio.renderers.default = "notebook"

# Create DataFrame
```

Vishal Acharya

```python
df = pd.DataFrame({
    "Department": ["IT", "HR", "Finance", "Marketing"],
    "Revenue": [120000, 80000, 150000, 95000]
})

# Create Bar Chart
fig = px.bar(df, x="Department", y="Revenue",
             title="Department-wise Revenue",
             color="Department")

fig.show()
```

Vishal Acharya



157 "Grouped Bar Chart: Compare yearly sales across regions.

Compare yearly sales across regions. import pandas as pd import plotly.express as px import plotly.io as pio

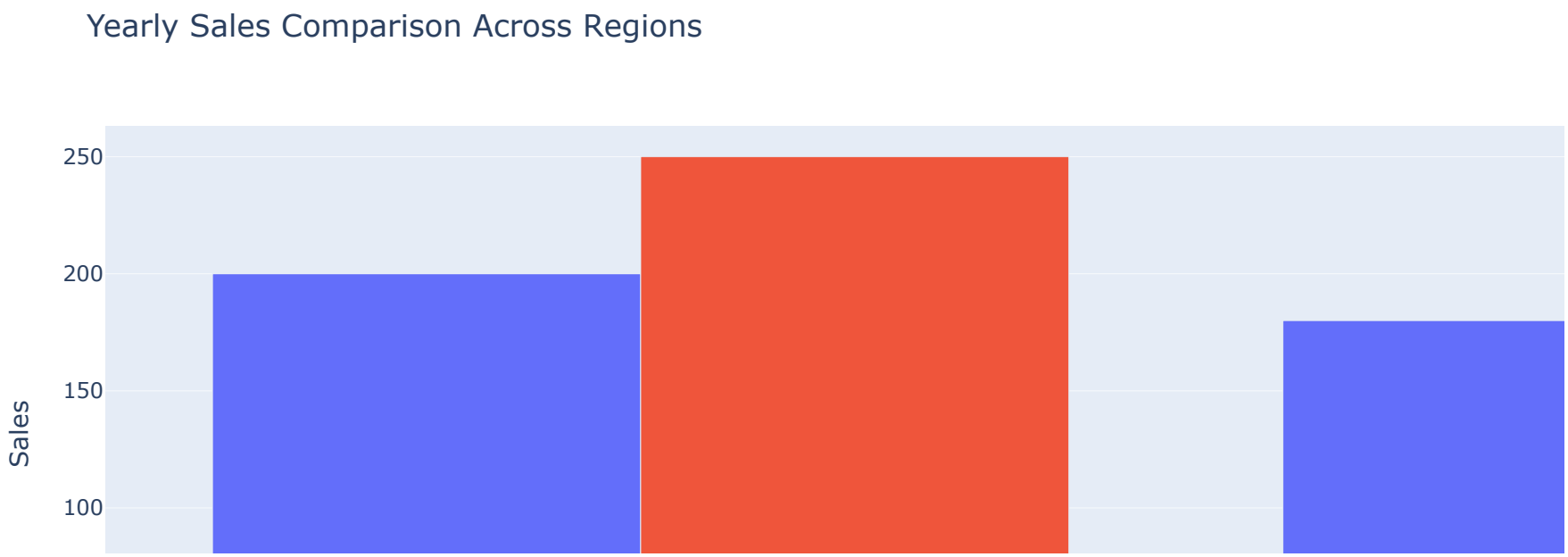
pio.renderers.default = ""notebook""

df = pd.DataFrame({ ""Region"": [""North"",""North"",""South"",""South""], ""Year"": [""2023"",""2024"",""2023"",""2024""], ""Sales"": [200,250,180,220] }) "

In [7]:
```
import pandas as pd
import plotly.express as px
import plotly.io as pio

pio.renderers.default = "notebook"

# Create DataFrame
df = pd.DataFrame({
    "Region": ["North","North","South","South"],
    "Year": ["2023","2024","2023","2024"],
    "Sales": [200,250,180,220]
})

# Grouped Bar Chart
fig = px.bar(df, x="Region", y="Sales",
             color="Year", barmode="group",
             title="Yearly Sales Comparison Across Regions")

fig.show()
```

Vishal Acharya

Vishal Acharya



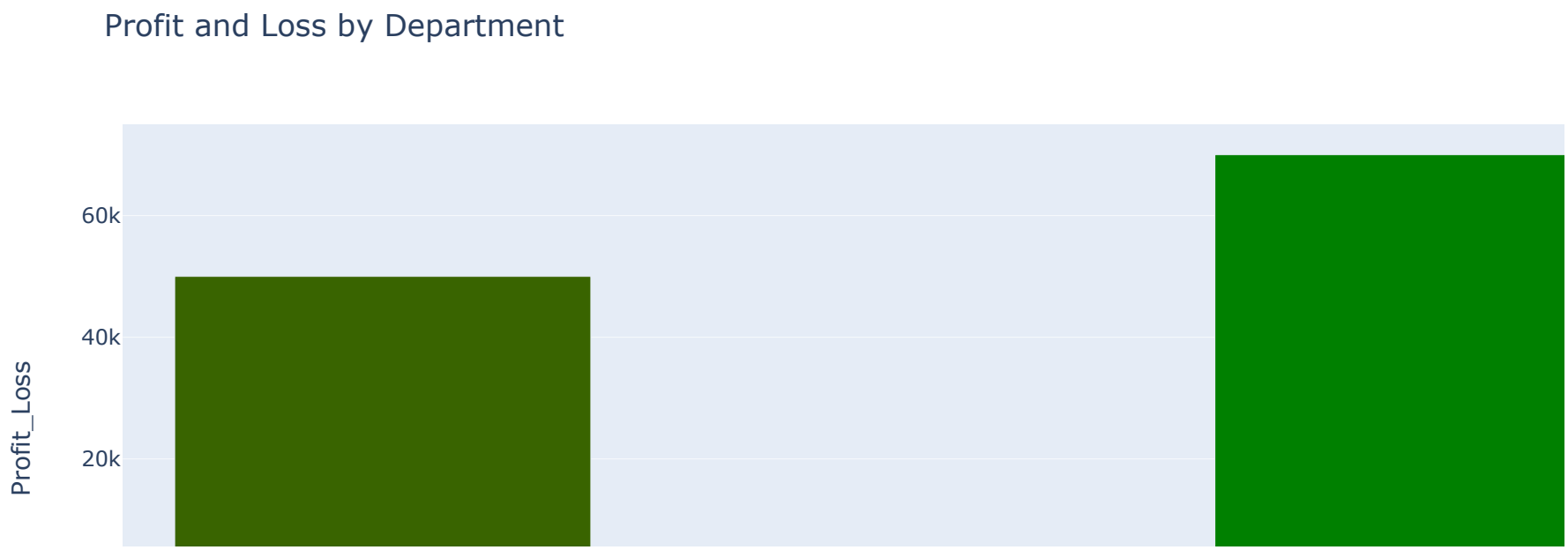
158 "Diverging Bar Chart: Display profit and loss of departments.

import pandas as pd import plotly.express as px import plotly.io as pio pio.renderers.default = ""notebook"" df = pd.DataFrame({ ""Department"": [""IT"",""HR"",""Finance"",""Marketing""], ""Profit_Loss"": [50000,-20000,70000,-15000] }) "

Vishal Acharya

In [8]:
```
import pandas as pd
import plotly.express as px
import plotly.io as pio

pio.renderers.default = "notebook"

# Create DataFrame
df = pd.DataFrame({
    "Department": ["IT","HR","Finance","Marketing"],
    "Profit_Loss": [50000,-20000,70000,-15000]
})

# Diverging Bar Chart
fig = px.bar(df, x="Department", y="Profit_Loss",
             color="Profit_Loss",
             color_continuous_scale=["red","green"],
             title="Profit and Loss by Department")

fig.show()
```

Vishal Acharya



159 "2D Group Scatter: Analyze relationship between Height and Weight grouped by Gender.

import pandas as pd import plotly.express as px import plotly.io as pio

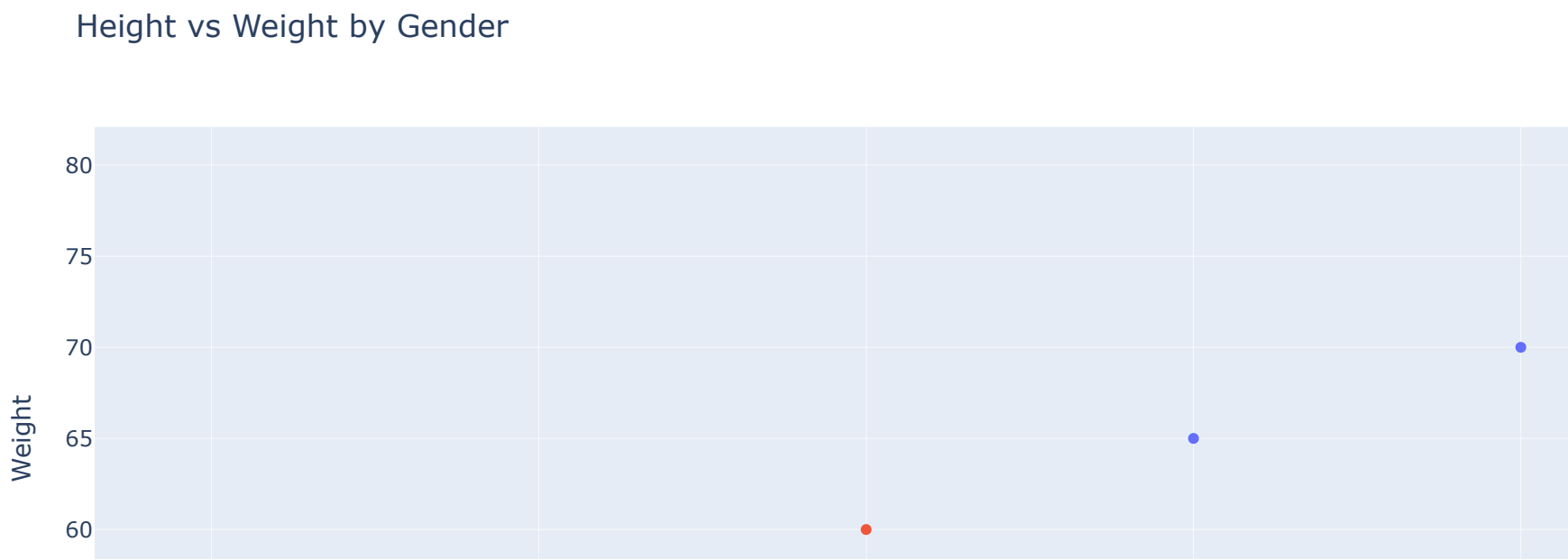
pio.renderers.default = ""notebook""

df = pd.DataFrame({ ""Height"":[150,160,170,180,165], ""Weight"":[50,60,70,80,65], ""Gender"":[""M"",""F"",""M"",""F"",""M""] }) "

In [9]:
```
import pandas as pd
import plotly.express as px
import plotly.io as pio

pio.renderers.default = "notebook"

# Create DataFrame
df = pd.DataFrame({
    "Height":[150,160,170,180,165],
    "Weight":[50,60,70,80,65],
    "Gender":["M","F","M","F","M"]
})

# Scatter Plot
fig = px.scatter(df, x="Height", y="Weight",
                 color="Gender",
                 title="Height vs Weight by Gender")

fig.show()
```

Vishal Acharya

Vishal Acharya



160 "Bubble Scatter:Sales vs Profit where bubble size represents Quantity.

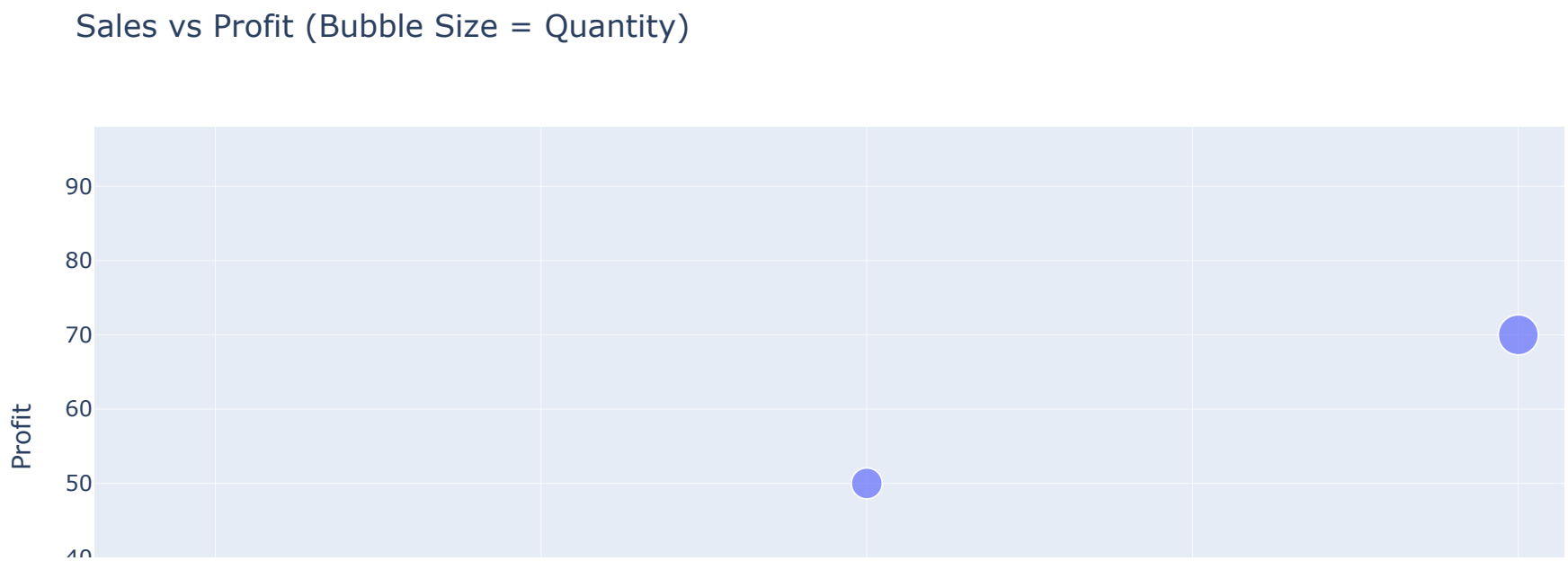
df = pd.DataFrame({ ""Sales"":[100,200,300,400], ""Profit"":[20,50,70,90], ""Quantity"":[10,30,50,80] })"

In [10]:
```
import pandas as pd
import plotly.express as px
import plotly.io as pio
```

Vishal Acharya

```python
pio.renderers.default = "notebook"

# Create DataFrame
df = pd.DataFrame({
    "Sales":[100,200,300,400],
    "Profit":[20,50,70,90],
    "Quantity":[10,30,50,80]
})

# Bubble Scatter Plot
fig = px.scatter(df, x="Sales", y="Profit",
                 size="Quantity",
                 title="Sales vs Profit (Bubble Size = Quantity)")

fig.show()
```

Vishal Acharya



161 "3D Scatter:Visualize 3D coordinates with color grouping.

df = pd.DataFrame({ ""X"":[1,2,3,4], ""Y"":[10,20,30,40], ""Z"":[5,15,25,35], ""Category"":[""A"",""B"",""A"",""B""] })"

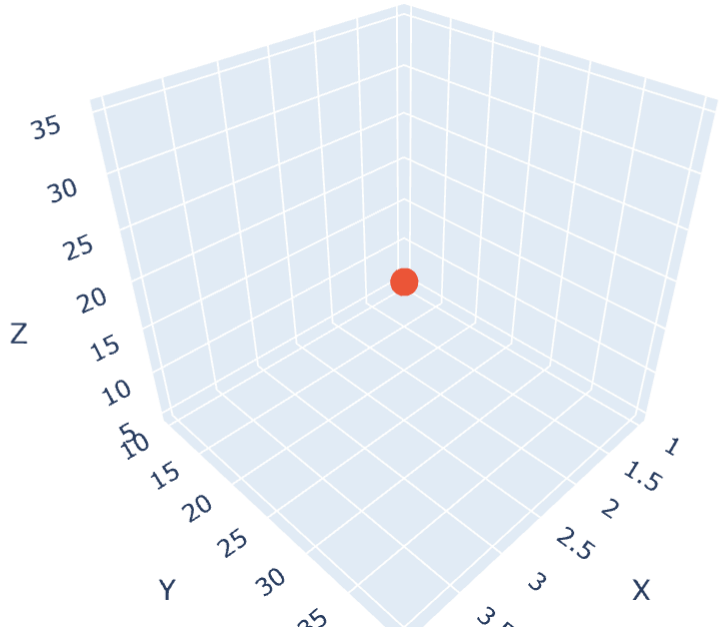
In [11]:
```
import pandas as pd
import plotly.express as px
```

Vishal Acharya

```python
import plotly.io as pio

pio.renderers.default = "notebook"

# Create DataFrame
df = pd.DataFrame({
    "X":[1,2,3,4],
    "Y":[10,20,30,40],
    "Z":[5,15,25,35],
    "Category":["A","B","A","B"]
})

# 3D Scatter Plot
fig = px.scatter_3d(df,
                    x="X", y="Y", z="Z",
                    color="Category",
                    title="3D Scatter Plot with Category Grouping")

fig.show()
```

Vishal Acharya

3D Scatter Plot with Category Grouping



162 "single Line Chart:Show yearly company growth.

df = pd.DataFrame({ ""Year"":[2020,2021,2022,2023,2024], ""Revenue"":[100,150,200,250,300] })"

Vishal Acharya

In [12]:
```
import pandas as pd
import plotly.express as px
import plotly.io as pio

pio.renderers.default = "notebook"

# Create DataFrame
df = pd.DataFrame({
    "Year":[2020,2021,2022,2023,2024],
    "Revenue":[100,150,200,250,300]
})

# Line Chart
fig = px.line(df, x="Year", y="Revenue",
              title="Yearly Company Growth",
              markers=True)

fig.show()
```

Vishal Acharya

163 "line Chart with Color Group: Track product sales trends.

df = pd.DataFrame({ ""Month"":[""Jan"",""Feb"",""Mar"",""Jan"",""Feb"",""Mar""], ""Sales"":[50,60,70,40,55,65], ""Product"": [""A"",""A"",""A"",""B"",""B"",""B""] })"

Vishal Acharya

In [13]:
```
import pandas as pd
import plotly.express as px
import plotly.io as pio

pio.renderers.default = "notebook"

# Create DataFrame
df = pd.DataFrame({
    "Month":["Jan","Feb","Mar","Jan","Feb","Mar"],
    "Sales":[50,60,70,40,55,65],
    "Product":["A","A","A","B","B","B"]
})

# Line Chart with Color Group
fig = px.line(df, x="Month", y="Sales",
              color="Product",
              markers=True,
              title="Product-wise Sales Trend")

fig.show()
```

Vishal Acharya

164 "Line Chart with Color Group: Track product sales trends.

df = pd.DataFrame({ ""Month"":[""Jan"",""Feb"",""Mar"",""Jan"",""Feb"",""Mar""], ""Sales"":[50,60,70,40,55,65], ""Product"": [""A"",""A"",""A"",""B"",""B"",""B""] })"

Vishal Acharya

In [14]:
```
import pandas as pd
import plotly.express as px
import plotly.io as pio

pio.renderers.default = "notebook"

df = pd.DataFrame({
    "Month":["Jan","Feb","Mar","Jan","Feb","Mar"],
    "Sales":[50,60,70,40,55,65],
    "Product":["A","A","A","B","B","B"]
})

fig = px.line(df, x="Month", y="Sales",
              color="Product",
              markers=True,
              title="Product Sales Trends")

fig.show()
```

Vishal Acharya

165 "Student Marks Distribution:

Create a dataset of 20 students containing their exam marks and grade category (A, B, C). Perform the following: Plot a histogram of Marks. Set nbins = 5. Use color = ""Grade"". Display values using text_auto = True. Add an appropriate title.

import pandas as pd import plotly.express as px import plotly.io as pio

Vishal Acharya

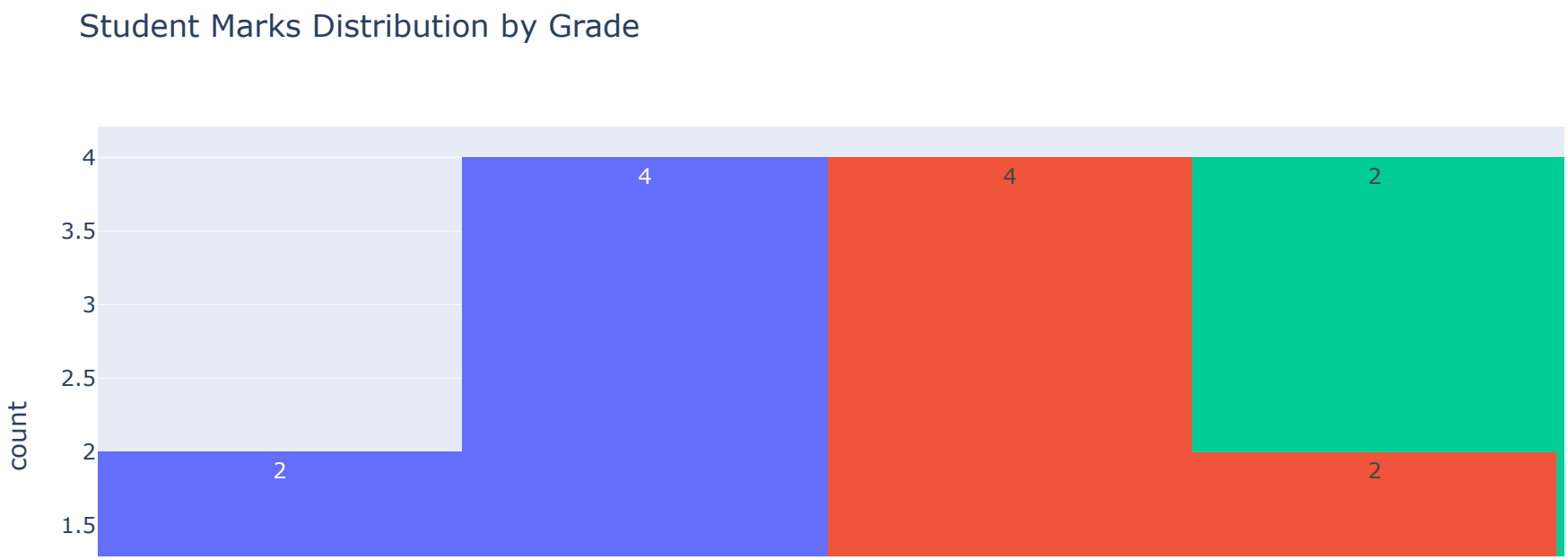
Set notebook renderer pio.renderers.default = ""notebook""

Create Dataset df = pd.DataFrame({ ""Student"": [f""S{i}"" for i in range(1, 21)], ""Marks"": [45, 50, 55, 60, 65, 70, 75, 80, 85, 90, 48, 52, 58, 62, 68, 72, 78, 82, 88, 92], ""Grade"": [""C"",""C"",""C"",""B"",""B"",""B"",""A"",""A"",""A"",""A"", ""C"",""C"",""C"",""B"",""B"",""B"",""A"",""A"",""A"",""A""] })
"

In [15]:

```
import pandas as pd
import plotly.express as px
import plotly.io as pio

# Set notebook renderer
pio.renderers.default = "notebook"

# Create Dataset
df = pd.DataFrame({
    "Student": [f"S{i}" for i in range(1, 21)],
    "Marks": [45, 50, 55, 60, 65, 70, 75, 80, 85, 90,
              48, 52, 58, 62, 68, 72, 78, 82, 88, 92],
    "Grade": ["C","C","C","B","B","B","A","A","A","A",
              "C","C","C","B","B","B","A","A","A","A"]
})

# Histogram
fig = px.histogram(df,
                   x="Marks",
                   color="Grade",
                   nbins=5,
                   text_auto=True,
                   title="Student Marks Distribution by Grade")

fig.show()
```

Vishal Acharya



166 "Employee Age Distribution:Create dataset of employee ages and departments.

Perform: Plot histogram of Age. Use color=""Department"". Use histfunc=""count"". Set nbins=5. Add proper title and width=700, height=400.

Vishal Acharya

import pandas as pd import plotly.express as px import plotly.io as pio

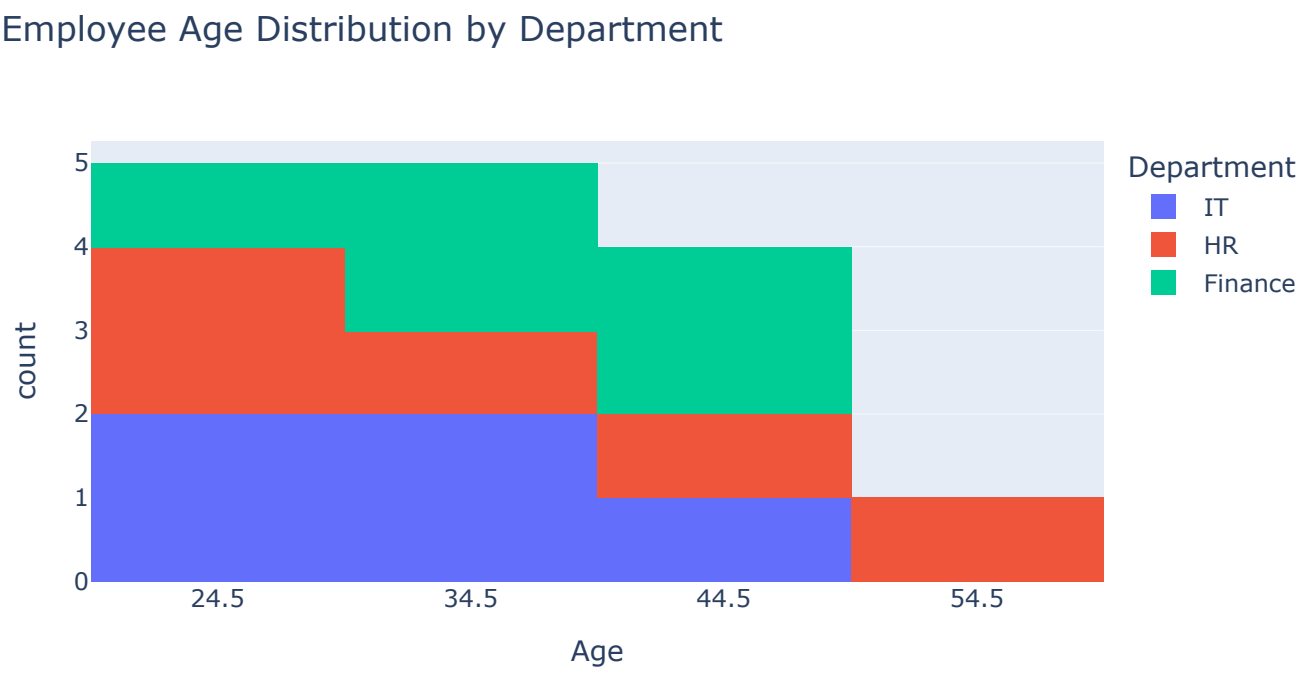
pio.renderers.default = ""notebook""

df = pd.DataFrame({ ""Age"":[22,25,28,30,35,40,45,50,29,33, 26,31,38,42,48], ""Department"":[""IT"",""IT"",""HR"",""HR"",""Finance"",""Finance"", ""IT"",""HR"",""Finance"",""IT"", ""HR"",""Finance"",""IT"",""HR"",""Finance""] }) "

In [16]:

```
import pandas as pd
import plotly.express as px
import plotly.io as pio

pio.renderers.default = "notebook"

# Create Dataset
df = pd.DataFrame({
    "Age":[22,25,28,30,35,40,45,50,29,33,
           26,31,38,42,48],
    "Department":["IT","IT","HR","HR","Finance","Finance",
                  "IT","HR","Finance","IT",
                  "HR","Finance","IT","HR","Finance"]
})

# Histogram
fig = px.histogram(df,
                   x="Age",
                   color="Department",
                   histfunc="count",
                   nbins=5,
                   title="Employee Age Distribution by Department",
                   width=700,
                   height=400)

fig.show()
```

Vishal Acharya



167 "Create dataset of monthly sales transactions (30 values) and Region (North, South).

Perform: Plot histogram of Sales. Use color=""Region"". Use pattern_shape=""Region"". Set nbins=6. Add title and custom color sequence.

import pandas as pd import plotly.express as px import plotly.io as pio

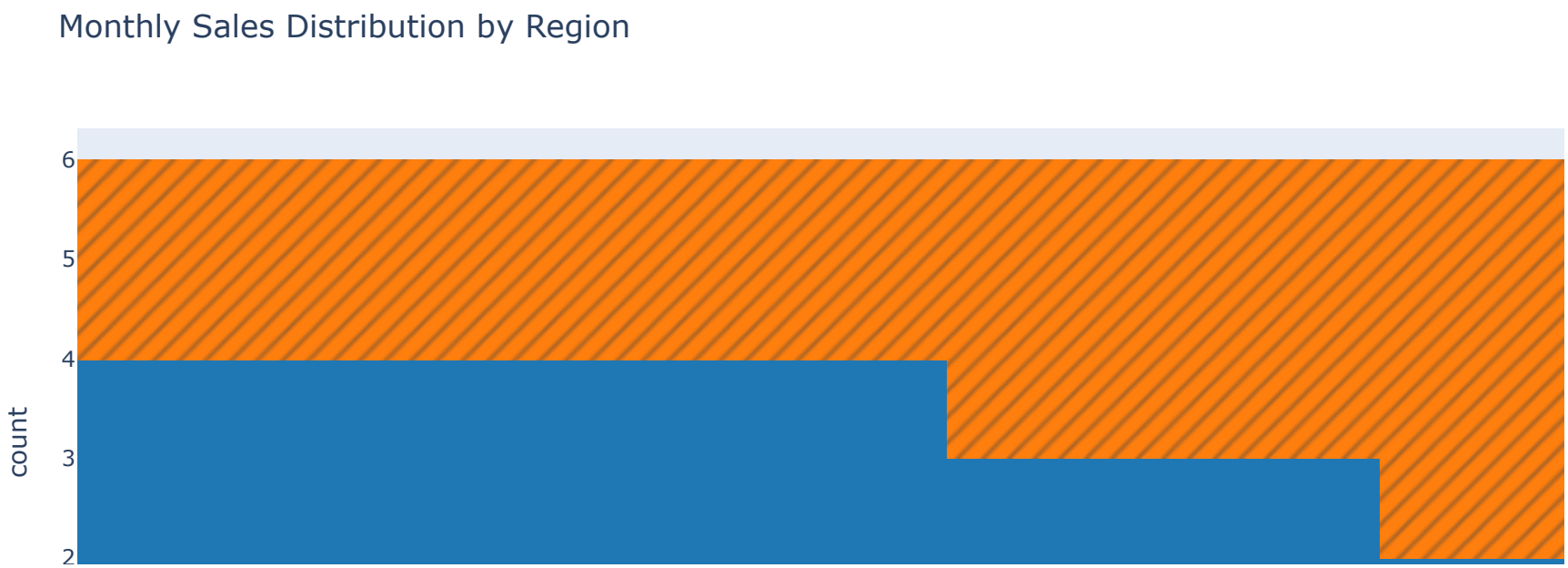
pio.renderers.default = ""notebook""

df = pd.DataFrame({ ""Sales"":[100,120,150,180,200,220,250,270,300,320, 110,130,160,190,210,230,260,280,310,330, 105,125,155,185,205,225,255,275,305,325], ""Region"":[""North""]*15 + [""South""]*15 })"

Vishal Acharya

In [17]:
```
import pandas as pd
import plotly.express as px
import plotly.io as pio

pio.renderers.default = "notebook"

# Create Dataset
df = pd.DataFrame({
    "Sales":[100,120,150,180,200,220,250,270,300,320,
             110,130,160,190,210,230,260,280,310,330,
             105,125,155,185,205,225,255,275,305,325],
    "Region":["North"]*15 + ["South"]*15
})

# Histogram
fig = px.histogram(df,
                   x="Sales",
                   color="Region",
                   pattern_shape="Region",
                   nbins=6,
                   color_discrete_sequence=["#1f77b4", "#ff7f0e"],
                   title="Monthly Sales Distribution by Region")

fig.show()
```

Vishal Acharya



168 "Product Market Share:Create dataset showing sales percentage of 5 products.

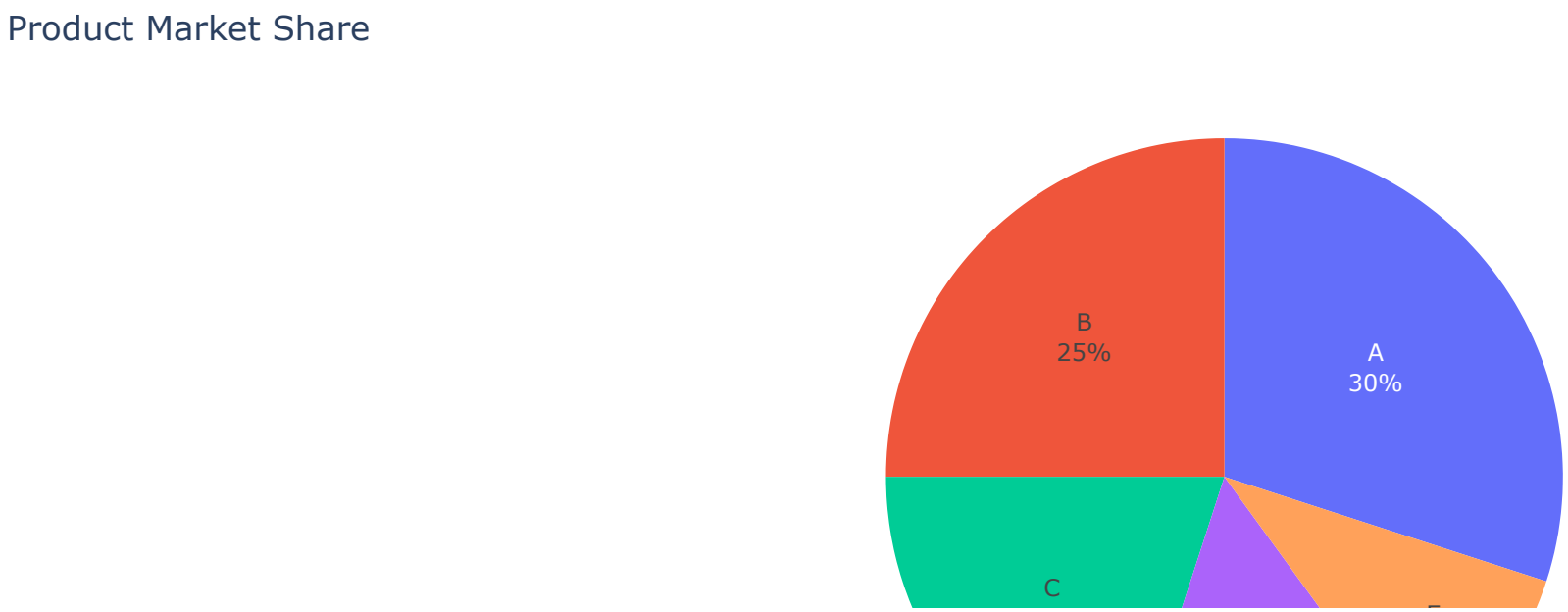
Perform: Plot pie chart using px.pie(). Show percentage and label. Use custom color sequence. Add title. import pandas as pd import plotly.express as px import plotly.io as pio pio.renderers.default = ""notebook"" df = pd.DataFrame({ ""Product"":[""A"",""B"",""C"",""D"",""E""],

""Sales"":[30,25,20,15,10] })"

Vishal Acharya

In [19]:
```
import pandas as pd
import plotly.express as px
import plotly.io as pio

pio.renderers.default = "notebook"

# Create Dataset
df = pd.DataFrame({
    "Product":["A","B","C","D","E"],
    "Sales":[30,25,20,15,10]
})

# Pie Chart
fig = px.pie(df,
             names="Product",
             values="Sales",
             title="Product Market Share",
             color_discrete_sequence=["#636EFA","#EF553B","#00CC96","#AB63FA","#FFA15A"])

# Show percentage + label
fig.update_traces(textinfo="label+percent")

fig.show()
```

Vishal Acharya



169 "Department Budget Allocation:

Create dataset of department budget allocation. Perform: Plot pie chart. Pull the highest budget department outward. Show percentage and label. Set text position inside.

import pandas as pd import plotly.express as px import plotly.io as pio

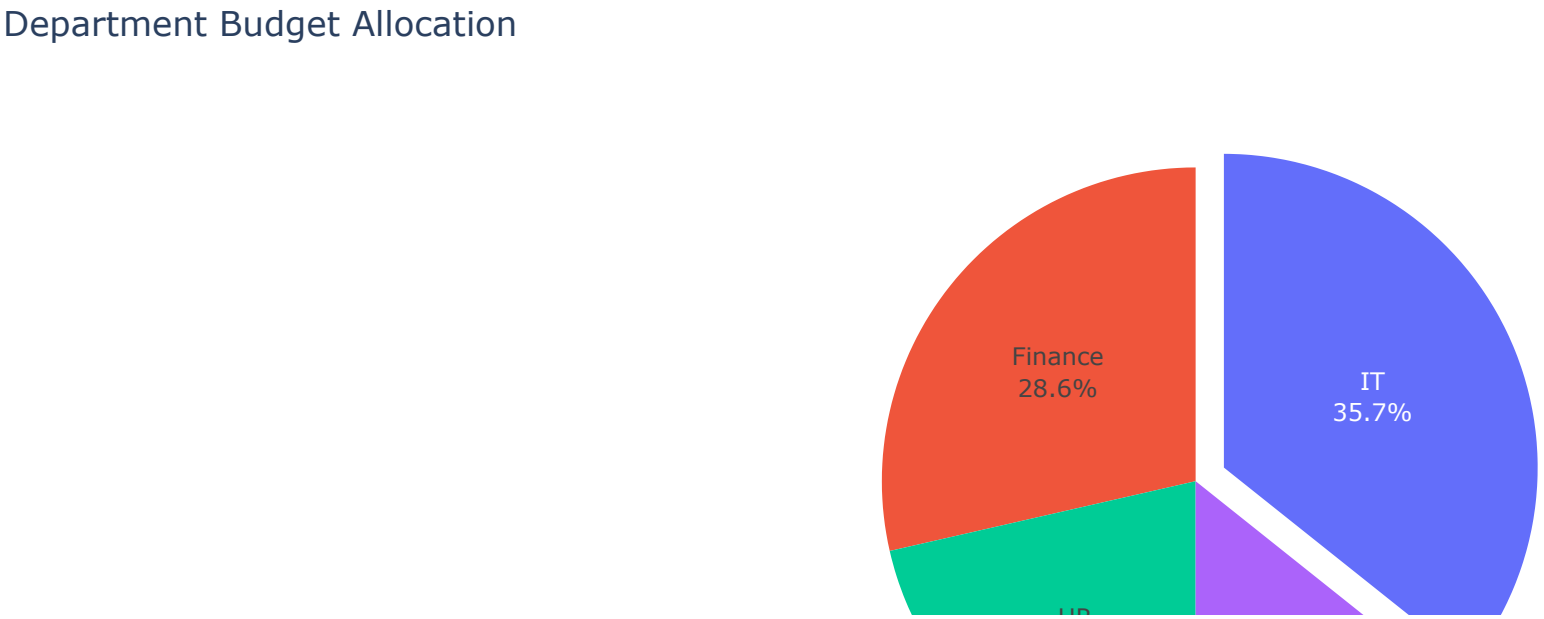
Vishal Acharya

pio.renderers.default = ""notebook""

df = pd.DataFrame({ ""Department"":[""IT"",""HR"",""Finance"",""Marketing""], ""Budget"":[50000,30000,40000,20000] }) "

In [20]:
```
import pandas as pd
import plotly.express as px
import plotly.io as pio

pio.renderers.default = "notebook"

# Create Dataset
df = pd.DataFrame({
    "Department":["IT","HR","Finance","Marketing"],
    "Budget":[50000,30000,40000,20000]
})

# Identify highest budget index
max_index = df["Budget"].idxmax()

# Create pull list
pull = [0.1 if i == max_index else 0 for i in range(len(df))]

# Pie Chart
fig = px.pie(df,
             names="Department",
             values="Budget",
             title="Department Budget Allocation",
             color_discrete_sequence=["#636EFA","#EF553B","#00CC96","#AB63FA"])

# Apply pull + text settings
fig.update_traces(pull=pull,
                  textinfo="label+percent",
                  textposition="inside")

fig.show()
```

Vishal Acharya



170 "Population Share by City:Create dataset of population of 4 cities.

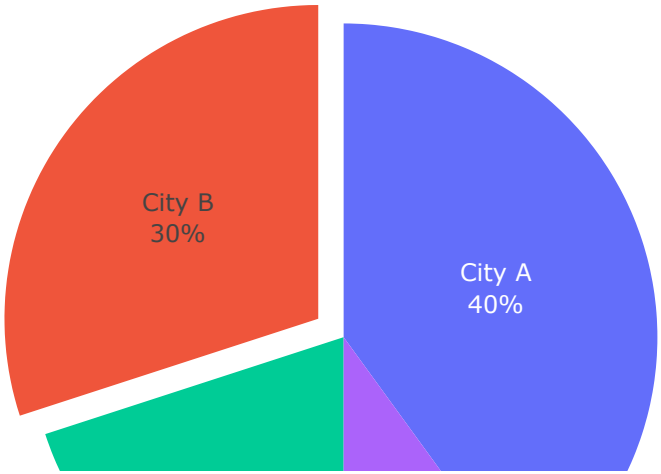
Perform: Plot pie chart. Use custom color sequence. Pull second slice outward. Display percent and label.import pandas as pd import plotly.express as px import plotly.io as pio

pio.renderers.default = ""notebook""

df = pd.DataFrame({ ""City"":[""City A"",""City B"",""City C"",""City D""], ""Population"":[400,300,200,100] }) "

In [21]:
```
import pandas as pd
import plotly.express as px
import plotly.io as pio

pio.renderers.default = "notebook"

# Create Dataset
df = pd.DataFrame({
    "City":["City A","City B","City C","City D"],
    "Population":[400,300,200,100]
})

# Pull second slice (City B)
pull = [0, 0.1, 0, 0]

# Pie Chart
fig = px.pie(df,
             names="City",
             values="Population",
             title="Population Share by City",
             color_discrete_sequence=["#636EFA","#EF553B","#00CC96","#AB63FA"])

# Update traces
fig.update_traces(pull=pull,
                  textinfo="label+percent")

fig.show()
```

Vishal Acharya

Vishal Acharya

Population Share by City